

Individuelle praktische Arbeit

EcoHub Testautomation und Mapping

21. Mai 2025



Version: 0.9

Laufzeit: 06.05.2025 - 21.05.2025

Kandidat

Kento Puleo
p.kento@bluewin.ch

Berufsbildner

Sascha Listner
sascha.listner@pax.ch

Fachkraft

Kushtrim Mjeku
kushtrim.mjeku@pax.ch

Hauptexperte

Janick Wüthrich
janick.wuethrich@gmail.com

Nebenexperte

Ralf Horstmöller
ralf.horstmoeller@roche.com

DOKUMENTENVERWALTUNG

Autor: Kento Puleo
Version: 0.9
Datum: 21. Mai 2025
Status: In Bearbeitung
Dateiname: ipa_kp_pax_2025.pdf

Version	Datum	Änderung
0.0.1	30.04.2025	Initialisierung des $\text{\LaTeX}$ Dokuments
0.1.0	06.05.2025	Definition der Arbeitspakete und Projektmethode
0.2.0	07.05.2025	Dokumentation der Analysephase, Anforderungen, Versionierung
0.3.0	08.05.2025	Abschluss der Analysephase, Beginn Designphase, Fehlerbehebung im LaTeX
0.4.0	09.05.2025	Überarbeitung Zeitplan, Dokumentation Testfälle
0.5.0	13.05.2025	Beginn der Implementierung, dokumentierung von Mapping-Konfiguration und LdapProperties
0.6.0	14.05.2025	Umsetzung und Dokumentation der Gruppen-Mapping-Logik, Start API-Testsetup
0.7.0	15.05.2025	Dokumentation der Implementierung der API-Tests, Start der CI/CD-Pipeline, Heartbeat eingeführt
0.8.0	16.05.2025	CI/CD-Pipeline Fehleranalyse und Korrektur, erfolgreiche Testausführung dokumentiert
0.9.0	20.05.2025	Dokumentation Qualitätssicherung, Verfassen Kapitel „Ergebnisse & Ausblick“
0.8.0	21.05.2025	Abschlussarbeiten, Finalisierung der Projektdokumentation

Inhaltsverzeichnis

I	Obligatorischer Teil	1
1	Aufgabenstellung im Originaltext	2
1.1	Titel der Arbeit	2
1.2	Ausgangslage	2
1.3	Detaillierte Aufgabenstellung	3
1.3.1	Mapping-Funktion	3
1.3.2	Testautomatisierung	3
1.4	Mittel und Methode	4
1.4.1	Entwicklungsumgebung	4
1.4.2	Versionierungsverwaltung	4
1.5	Vorkenntnisse	4
1.5.1	Frameworks und Libraries	4
1.5.2	Programmiersprachen	5
1.5.3	Versionierungsverwaltung	5
1.6	Vorarbeiten	5
1.7	Neue Lerninhalte	5
1.7.1	Unternehmen	5
1.7.2	Persönlich	5
1.8	Arbeiten in den letzten sechs Monaten	5
1.9	Individuelle Kriterien	6
2	Projektmethode	7
2.1	Interne Projektmethode	7
2.2	Gewählte Projektmethode	7
2.2.1	Die Wasserfallmethode	7
2.2.2	Die Phasen der Wasserfallmethode genauer erklärt	8
2.3	Begründung der Wahl	9
2.4	Vergleich mit agilen Methoden	9
3	Projektaufbauorganisation	11
4	Arbeitspakete	12

5 Backupkonzept	14
6 Zeitplan	17
7 Persönliches Fazit	20
8 Arbeitsjournal	21
 II Projektdokumentation	 22
9 Kurzfassung	23
9.1 Ausgangslage	23
9.2 Zielsetzung	23
9.3 Ergebnis	23
10 Initialisierung/Analyse	24
10.1 Erweiterung der Ausgangslage	24
10.1.1 IST-Stand	24
10.1.2 LDAP-Verzeichnisstruktur	25
10.1.3 Business Process Model der updateUser-Funktion (BPM)	27
10.1.4 Tabelle der zu testenden Endpunkte	27
10.1.5 Struktur der SOAP-Nachrichten	28
10.1.6 Analyse der bestehenden CI/CD-Pipeline	30
10.1.7 Darstellung der Systemstruktur im IST-Stand	31
10.1.8 Problemanalyse	35
10.1.9 Chancen und Risiken	35
10.1.10 Zielsetzung	36
10.1.11 Abgrenzung	36
10.2 Erweiterte Aufgabenstellung	36
10.3 Anforderungen	36
10.3.1 Funktionale Anforderungen	36
10.3.2 Nicht-funktionale Anforderungen	37
10.4 Rahmenbedingungen und Annahmen	38
11 Planung/Design	39
11.1 Zielbild und Anforderungen	39
11.2 Komponentendesign: Erweiterung von updateUser	40
11.2.1 Konfigurationsdesign: YAML-Mapping	40
11.3 Testdesign und API-Abdeckung	41
11.4 Testfälle	42
11.4.1 Vollständige XML-Payloads zu addUser, updateUser	42

11.4.2 Endpoint: addUser	43
11.4.3 Endpoint: updateUser	47
11.4.4 Endpoint: deleteUser	50
11.4.5 Endpoint: getUserStatus	53
11.4.6 Endpoint: addUser, updateUser, deleteUser, getUser	55
11.4.7 Erweiterung der CI/CD-Pipeline mit Docker und Playwright	55
11.5 Technische Arbeitspakete	56
11.6 Tools und Technologien	57
11.6.1 Testen der Mapping-Funktion mit SoapUI und OpenLDAP	57
11.6.2 Automatisierte API-Tests mit Playwright	58
11.6.3 Versionierung	58
11.7 Clean Code Prinzipien und Namenskonzept	60
12 Implementierung	62
12.1 Mapping-Funktion	62
12.1.1 Konfigurationsdatei	62
12.1.2 Entfernen des Nutzers aus bestehenden Gruppen	63
12.1.3 Hinzufügen des Nutzers zu den neuen Gruppen	66
12.1.4 Einbindung der Mapping-Funktionen in updateUser	70
12.2 Implementierung der Tests	71
12.2.1 Einrichtung und Konfiguration des Testframeworks	71
12.2.2 Generierung der SOAP-Requests	71
12.2.3 Test: addUser	73
12.2.4 Test: updateUser	77
12.2.5 Test: deleteUser	80
12.2.6 Test: getUserStatus	83
12.2.7 Automatisiertes Zurücksetzen der LDAP-Testumgebung	84
12.2.8 Automatisiertes Zurücksetzen der LDAP-Testumgebung	85
12.2.9 Verfügbarkeit der Applikation prüfen – globalSetup.ts	88
12.3 Implementierung der CI/CD Pipeline	89
12.3.1 Erweiterung der CI/CD-Pipeline zur Ausführung automatisierter API-Tests	89
13 Test/Qualitätssicherung	95
13.1 Test der Mapping-Funktion	95
13.1.1 Erster Test der Mapping-Funktion EL	95
13.1.2 Zweiter Test der Mapping-Funktion KL	98
13.1.3 Unit-Test der Mapping-Funktion	100
13.2 API-Tests mit Playwright	101
13.3 CI/CD-Pipeline und Testintegration	103
13.3.1 Fehlermeldung bei nicht erreichbarem Service	104
13.3.2 Verifikation des Cronjobs	105

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



13.3.3 Hinweis zur README-Ergänzung	105
14 Ergebnisse & Ausblick	107
14.1 Ergebnisse der Umsetzung	107
14.2 Ausblick und Weiterentwicklung	108
 III Anhang	 109
Hilfsmittelverzeichnis	110
Abkürzungsverzeichnis	111
Glossar	112
Abbildungsverzeichnis	113
Tabellenverzeichnis	115
Quellenverzeichnis	116

Teil I

Obligatorischer Teil

KAPITEL 1

AUFGABENSTELLUNG IM ORIGINALTEXT

Im Folgenden Kapitel wird die Detaillierte Aufgabenstellung aus dem PkOrg kopiert und formatiert, eine erweiterte Version kann im zweiten Teil der Dokumentation gefunden werden (9.1 Erweiterte Ausgangslage & 9.2 Erweiterte Aufgabenstellung).

1.1 Titel der Arbeit

Implementierung von Mapping-Funktionen und Testautomatisierung-API im EcoHub-Service

1.2 Ausgangslage

EcoHub ist eine Plattform, die die Zusammenarbeit zwischen Makler/Brokern und Versicherungen erleichtert. Seitens Pax nutzen wir dabei die Funktionen Single-Sign-On (SSO) und User-Provisioning. Makler melden sich bei EcoHub an und können über diese Plattform auf verschiedene Versicherungen zugreifen, mit denen Sie eine Service-Vereinbarung haben.

Das geschieht folgendermassen:

1. Die Pax ist als Nutzer bei EcoHub angemeldet.
2. Broker, privat oder einer Organisation, melden sich auch als Nutzer auf EcoHub an.
3. Der Broker und die Pax können eine Service-Vereinbarung treffen. Dazu müssen beide Parteien der Vereinbarung zustimmen.
4. Nach der Vereinbarung werden die Broker Daten an die Pax gesendet. Diese prüft, ob die E-Mail des Brokers gespeichert und bekannt ist.

5. Sollte die E-Mail bekannt sein, wird eine IDP-Nummer von EcoHub angefragt. Diese wird dem Benutzer zugeteilt.
6. Sollte die E-Mail nicht bekannt sein, wirft die Pax eine Fehlermeldung an EcoHub.
7. Nachdem die IDP-Nummer dem Broker zugeteilt wurde, kann dieser über die Plattform auf die Pax Seite zugreifen, ohne sich erneut anmelden zu müssen.

1.3 Detaillierte Aufgabenstellung

Im Rahmen der IPA werden zwei Aspekte des EcoHub-Services weiterentwickelt, um dessen Funktionalität und Zuverlässigkeit zu verbessern. Einerseits wird eine Mapping-Funktion implementiert, die die automatische und präzise Zuordnung von Nutzern zu Applikationsgruppen ermöglicht. Andererseits wird eine umfassende Testautomatisierung auf API-Ebene eingeführt, um die Qualität der Schnittstellen systematisch zu prüfen und Fehler frühzeitig zu erkennen.

Beide Massnahmen zielen darauf ab, den Betrieb des EcoHub-Services effizienter zu gestalten, die Wartung zu vereinfachen und eine stabile Systemumgebung zu gewährleisten. Die nachfolgenden Abschnitte erläutern die Details zu den einzelnen Implementierungen.

1.3.1 Mapping-Funktion

Die Mapping-Funktion sorgt dafür, dass neu angelegte und bestehende Nutzer ihrer Applikationsgruppen zugeordnet werden. Diese Zuordnungen werden im OpenLDAP gespeichert und enthalten die eindeutige Kennnummer jedes Nutzers. So wird eine Präzise und nachhaltige Zuordnung gewährleistet.

Testkonzept der Mapping-Funktion

Um die Funktionalität der Mapping-Funktion sicherzustellen, wird diese umfassend getestet. Dazu gehören:

- Unit-Tests, um einzelne Komponenten isoliert zu prüfen
- Component-Tests, um die Zusammenarbeit der Module zu evaluieren
- Integration-Tests, die die Funktionalität im Gesamtsystem überprüfen

Diese Teststufen gewährleisten, dass die Mapping-Funktion fehlerfrei und stabil im Produktivbetrieb läuft.

1.3.2 Testautomatisierung

API-Tests

Der EcoHub-Service umfasst vier Schnittstellen, die künftig durch Playwright-API-Tests systematisch geprüft werden. Ziel ist es, Fehler frühzeitig zu erkennen und die Zuverlässigkeit der Schnittstellen sicherzustellen. Bei

fehlschlagenden Tests wird eine Benachrichtigung an die zuständigen Personen versendet, um eine zeitnahe Problembeseitigung zu gewährleisten.

Testumgebung

Die Durchführung der Tests erfolgt sowohl in der Testumgebung innerhalb der Continuous Deployment (CD) Pipeline als auch lokal. Für die lokale Ausführung wird ein embedded LDAP eingesetzt, das eine flexible und unabhängige Testumgebung ermöglicht.

Testdaten

Um die API-Tests zu unterstützen, werden spezifische Testdaten erstellt:

- Für das lokale Testen werden die Testdaten direkt im Repository hinterlegt
- In der CD-Pipeline wird zusätzlich ein Test-User im bestehenden LDAP-System angelegt, um die Funktionalität unter möglichst realistischen Bedingungen zu prüfen

1.4 Mittel und Methode

1.4.1 Entwicklungsumgebung

- | | |
|----------------------|-------------------------|
| • Lenovo ThinkPad X1 | Kenntnisse seit 2 Jahre |
| • Windows 11 | Kenntnisse seit 3 Jahre |
| • IntelliJ Ultimate | 3 Jahre |

1.4.2 Versionierungsverwaltung

- | | |
|----------|-------------------------|
| • GitHub | Kenntnisse seit 3 Jahre |
|----------|-------------------------|

1.5 Vorkenntnisse

1.5.1 Frameworks und Libraries

- | | |
|--------------|---------------------------|
| • Playwright | Kenntnisse seit 1/2 Jahre |
| • SpringBoot | Kenntnisse seit 2 Jahre |

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



1.5.2 Programmiersprachen

- TypeScript
- Java

Kenntnisse seit 3 Jahre

Kenntnisse seit 3 Jahre

1.5.3 Versionierungsverwaltung

- GitHub

Kenntnisse seit 3 Jahre

1.6 Vorarbeiten

Um eine gute und effiziente IPA durchführen zu können, wurden gewisse Vorarbeiten geleistet, welche es ermöglichte, Wissen und Erfahrungen zu gewinnen.

- Dokument Vorlage
- Einarbeitung in die Tools

1.7 Neue Lerninhalte

1.7.1 Unternehmen

Ziel ist es, Fehler frühzeitig zu erkennen und die Zuverlässigkeit der Schnittstellen sicherzustellen. Ausserdem soll die Applikationsgruppenzuweisung zukünftige Arbeiten erleichtern und den Nutzern die gewünschten Applikationen zur Verfügung stellen.

1.7.2 Persönlich

- Der Umgang und die Erfahrung Tests auf API-Ebene zuschreiben und Automatisieren
- Repetition und Erweiterung von Java und Springboot wissen

1.8 Arbeiten in den letzten sechs Monaten

Die letzten 6 Monate wurden in verschiedene Themen investiert. Folgend die Themen die behandelt wurden und die passende Zeitspanne.

- August & September: Vergleich und PoC RxJS zu Angular Signals
- Oktober & November: Testautomatisierung Playwright E2E Pilotversuch
- Dezember: Bekanntmachung mit IPA Thema

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



- Januar & Februar: Testautomatisierung API im Brokerportal

1.9 Individuelle Kriterien

Die individuellen Kriterien wurden vor der IPA definiert und von einem Validierungsexperten bestätigt. Um in dieses Dokument nicht unnötig in die Länge zu ziehen, wird im Anhang nochmals das Dokument zur Aufgabenstellung mitgegeben, darin ist auch eine Liste aller Kriterien zu finden.

KAPITEL 2

PROJEKTMETHODE

In diesem Kapitel wird die Wahl der Projektmethodik für die vorliegende Arbeit dargestellt. Zunächst wird auf die intern üblicherweise eingesetzte Projektmethode eingegangen. Anschliessend wird die für dieses Projekt gewählte Vorgehensweise vorgestellt und mit alternativen Methoden verglichen. Abschliessend wird die Entscheidung für die gewählte Methode begründet.

2.1 Interne Projektmethode

In den Softwareentwicklungsteams der Pax wird auf agile Projektmethoden gesetzt. Dabei kommt eine Kombination aus Scrum und Kanban zum Einsatz. Die Projekte werden in Sprints unterteilt, die wiederum in verschiedene Phasen gegliedert sind. Auch typische Scrum-Elemente wie Reviews und Retrospektiven werden regelmässig durchgeführt.

Zur Steuerung des Arbeitsprozesses wird jedoch ein Kanban-Board verwendet, um die Aufgaben zu visualisieren und den Fortschritt zu verfolgen. So lässt sich nachvollziehen, wer an welcher Aufgabe arbeitet und wie viel Zeit in jede einzelne Aufgabe investiert wird.

Ein Sprint dauert in der Regel etwa drei Wochen und beginnt mit einer Planungsphase. Es folgt eine Umsetzungsphase, die schliesslich von einer Testphase abgelöst wird. Am Ende eines Sprints erfolgt eine Reflexion, in der Erfahrungen und Erkenntnisse gesammelt sowie dokumentiert werden.

2.2 Gewählte Projektmethode

2.2.1 Die Wasserfallmethode

Bei der Wasserfallmethode handelt es sich um ein klassisches Projektmanagement-Modell, das besonders in der Softwareentwicklung weit verbreitet ist. Sie folgt einem linearen, sequenziellen Ansatz, bei dem das

Projekt in klar abgegrenzte Phasen unterteilt wird. Jede Phase muss vollständig abgeschlossen sein, bevor die nächste beginnt, wodurch der gesamte Projektverlauf strukturiert und gut planbar ist.

Das untenstehende Diagramm des Wasserfallmodells veranschaulicht diesen Ablauf deutlich: Es zeigt, wie die Phasen Anforderungsanalyse, Planung, Umsetzung, Test und Abschluss nacheinander durchlaufen werden. In der Regel erfolgt der Fortschritt nur in eine Richtung - ähnlich wie Wasser, das einen Wasserfall hinabfließt. Dennoch kann es in Ausnahmefällen notwendig sein, zu einer vorherigen Phase zurückzukehren, etwa um entdeckte Fehler oder Fehleinschätzungen zu korrigieren.

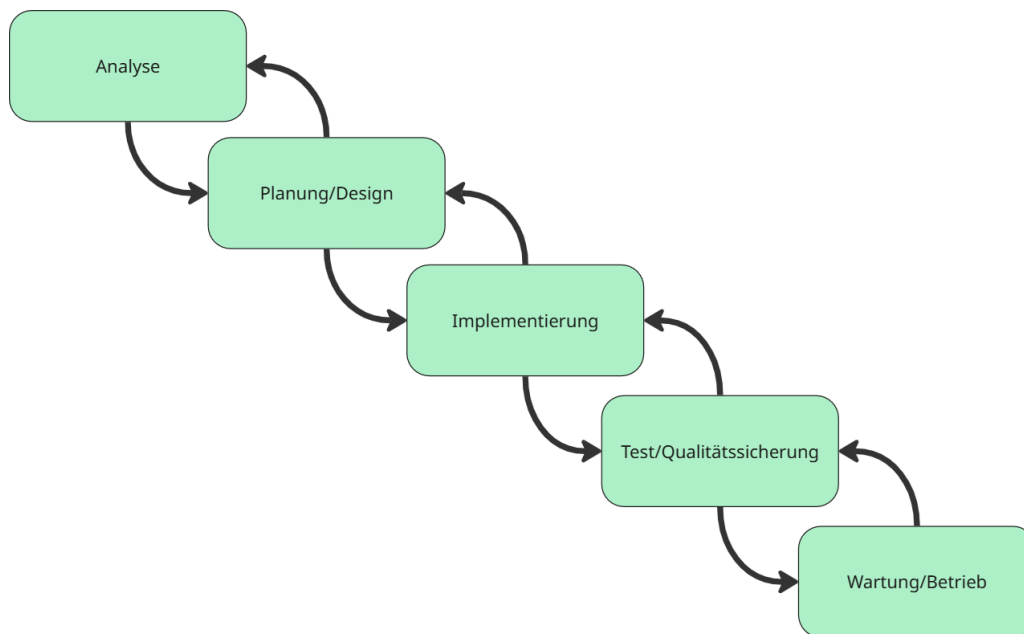


Abbildung 2.1: Wasserfallmodell (Quelle: Eigenerarbeitung)

2.2.2 Die Phasen der Wasserfallmethode genauer erklärt

Analyse

In dieser initialen Phase werden sämtliche funktionalen und nicht-funktionalen Anforderungen erhoben und dokumentiert. Gemeinsam mit allen Stakeholdern werden diese dokumentiert, der Projektumfang definiert und die Ziele festgelegt.

Planung/Design

Auf Basis der Anforderungen wird die Systemarchitektur konzipiert. Dabei werden alle Komponenten und ihre Schnittstellen definiert, die zu verwendenden Technologien und Frameworks festgelegt sowie Datenmodelle und Datenbankschemas erstellt.

Implementierung

In der Implementierungsphase erfolgt die eigentliche Programmierung der Software gemäß der Design-

Spezifikation. Die Entwickler erstellen Code nach definierten Standards, implementieren Unit-Tests und dokumentieren den Quellcode entsprechend der Vorgaben.

Test/Qualitätssicherung

Nach der Fertigstellung der einzelnen Komponenten werden diese zusammengeführt und integriert. Durch umfangreiche Tests wird die Gesamtfunktionalität validiert. Auftretende Fehler werden behoben und die Leistung des Systems optimiert.

Betrieb/Wartung

In dieser Phase wird das entwickelte System dem Kunden vorgestellt und zur abnahme bereitgestellt. Bei internen entwicklungen wird die Funktion dem Auftraggeber, meist Buissnesengineer oder Productowner, vorgestellt. Nach der Vorstellung wird die Funktion in die nächste Umgebung deployed und von den entsprechenden Personen abgenommen.

2.3 Begründung der Wahl

Die Wasserfallmethode wurde gewählt, da die Anforderungen im Vorfeld bereits klar definiert wurden und sich sehr wahrscheinlich nicht ändern werden. Es handelt sich um ein kleines Projekt, welches von einer einzigen Person ausgeführt wird, da ist es wichtig, keine komplexe Methode zu verwenden, welche womöglich viel Zeit oder Strukturierung benötigt.

2.4 Vergleich mit agilen Methoden

Für dieses Projekt gibt es keine laufenden Rückmeldungen oder Feedbackzyklen, die eine agile Methode rechtfertigen würden. Wie in der Tabelle 2.1: Vergleich zwischen Wasserfall und agilen Methoden zu sehen ist.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Tabelle 2.1: Vergleich zwischen Wasserfall und agilen Methoden

Kriterium	Wasserfall	Agile Methoden
Projektgrösse	Ideal für kleine, überschaubare Projekte	Gut für grosse, komplexe Projekte
Anforderungsänderungen	Schwierig zu implementieren	Flexibel und einfach umsetzbar
Planung	Detaillierte Vorplanung notwendig	Iterative Planung möglich
Teamgrösse	Funktioniert gut mit Einzelpersonen	Optimiert für Teams
Dokumentation	Umfangreich von Anfang an	Wächst mit dem Projekt
Risikomanagement	Risiken werden früh identifiziert	Kontinuierliche Risikobewertung

KAPITEL 3

PROJEKTAUFBAUORGANISATION

Im folgenden Kapitel wird die Projektaufbauorganisation veranschaulicht. Sie beschreibt die strukturierte Verteilung von Rollen, Aufgaben und Verantwortlichkeiten innerhalb des Projekts. Eine klar definierte Aufbauorganisation ist entscheidend für einen reibungslosen Ablauf, eine effiziente Kommunikation sowie die zielgerichtete Umsetzung der Projektziele.

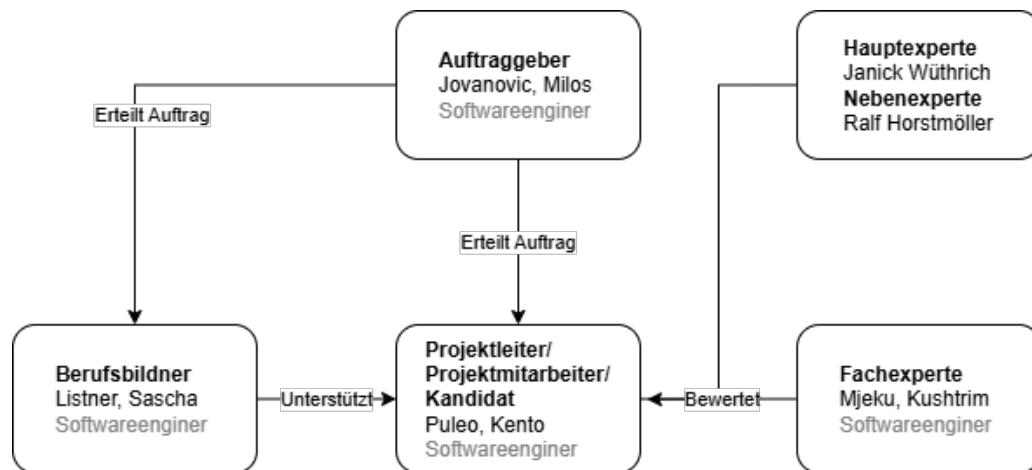


Abbildung 3.1: Projektaufbauorganisation (Quelle: Eigenerarbeitung)

In der oberen Grafik ist zu sehen welche Rolle die beteiligten Person in diesem Projekt erfüllen.

Der Auftrag wurde von einem Senior-Entwickler mit Projekterfahrung im Bereich EcoHub erteilt. Die fachliche Betreuung des Kandidaten erfolgt durch den Berufsbildner sowie einen internen Fachexperten, die ihn bereits seit etwa einem Jahr begleiten. Bei technischen Fragen steht der Berufsbildner als erste Ansprechperson zur Verfügung. Die Bewertung der Arbeit erfolgt zunächst durch den internen Fachexperten, anschliessend durch einen Haupt- und einen Nebenexperten, die extern besetzt sind.

KAPITEL 4

ARBEITSPAKETE

In diesem Kapitel werden die im Projekt definierten Arbeitspakete dokumentiert. Jedes Arbeitspaket umfasst eine eindeutige ID, einen Titel, eine Beschreibung sowie die geschätzte Bearbeitungsdauer. Die Arbeitspakete dienen der strukturierten Planung und bilden die Grundlage für die Umsetzung des Projekts. Eine Übersicht aller Arbeitspakete ist in Tabelle 4.1: Aufgabenübersicht dargestellt. Da es sich um ein Einzelprojekt handelt, erfolgt keine namentliche Zuweisung der Aufgaben.

Tabelle 4.1: Aufgabenübersicht

ID	Titel	Beschreibung	Dauer
1	Definition Arbeitspakete Grob	Erste Grobdefinition aller Arbeitspakete zur Strukturierung des Projekts.	2h
2	Erfassung der Arbeitspakete im Zeitplan v1	Erster Entwurf des Zeitplans mit allen Arbeitspaketen und Meilensteinen.	2h
3	Erarbeitung der Aufgabenstellung Originaltext	Import und Anpassung des Originaltexts aus PkOrg.	2h
4	Definition Projektmethode	Auswahl der Projektmethodik inkl. Begründung und Vergleich.	1h
5	Dokumentation Projektaufbauorganisation	Darstellung der Projektorganisation als Text und Grafik.	1h
6	Definition Arbeitspakete Detailliert	Verfeinerung der APs aus der Grobdefinition.	2h
7	Dokumentation Backupkonzept	Entwicklung und Dokumentation eines 3-2-1-1-0 Backupkonzepts.	2h
8	Erfassung der Arbeitspakete im Zeitplan v2	Finalisierung des Zeitplans mit SOLL-Version.	2h

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Fortsetzung von Tabelle 4.1

ID	Titel	Beschreibung	Dauer
9	Erleuterung und Erweiterung der Ausgangslage	Analyse der Problemstellung und IST-Situation.	1h
10	Erleuterung der Aufgabenstellung	Detaillierte Darstellung der Projektaufgabe.	1h
11	Anforderungen und Rahmenbedingungen definieren	Festlegung aller funktionalen und nicht-funktionalen Anforderungen.	2h
12	Planung der Erweiterung von updateUser	Konzeption der notwendigen Anpassungen an der bestehenden Logik.	2h
13	Konzept und Idee der API-Tests	Auswahl von Tools, Definition der Teststrategie.	4h
14	Definition & Dokumentation der Testfälle	Testfallspezifikation für alle vier Endpunkte.	4h
15	Dokumentation der Tools und Technologien	Beschreibung aller im Projekt verwendeten Technologien und Tools.	2h
16	Definition Clean Code Prinzipien	Festlegung der zu beachtenden Code-Prinzipien.	2h
17	Implementierung der Mapping-Funktion	Implementierung der Logik zur Rollenzuordnung in LDAP.	8h
18	Dokumentierung der Mapping-Funktion	Technische Beschreibung der Implementierung.	4h
19	Implementierung der API-Tests	Implementierung der automatisierten Playwright-Tests.	8h
20	Dokumentierung der API-Tests	Beschreibung des Testframeworks und der Testlogik.	4h
21	Implementierung der Pipeline	CI/CD Integration mit Tests, App und LDAP-Setup.	6h
22	Dokumentierung der Pipeline	Beschreibung der GitHub Actions und Abläufe.	2h
23	Testen der Mapping-Funktion & dokumentieren	Durchführung und Dokumentation der Unit Tests zur Mapping-Logik.	2h
24	Test und Durchlauf der API-Tests	Testdurchführung und Fehleranalyse automatisierter Tests.	2h
25	Testen der Pipeline & dokumentieren	Validierung der CI/CD-Abläufe und Protokollierung.	2h
26	Dokumentation der Ergebnisse	Zusammenfassung der Resultate des Projekts.	1h
27	Dokumentation des Ausblicks	Reflexion über mögliche Weiterentwicklungen.	1h
28	Dokument korrektur	Korrekturlesen und stilistische Überarbeitung.	6h

KAPITEL 5

BACKUPKONZEPT

Während dieser Arbeit kommt die moderne **3-2-1-1-0-Backup-Strategie** zum Einsatz. Diese Regel dient der langfristigen Sicherstellung der Datenverfügbarkeit und schützt insbesondere vor Datenverlust durch Hardwarefehler, menschliches Versagen oder Schadsoftware.

Grundlagen der 3-2-1-1-0-Regel

Die einzelnen Zahlen stehen für folgende Prinzipien:

3. Es existieren mindestens **drei Kopien** der Daten.
2. Die Kopien befinden sich auf **mindestens zwei unterschiedlichen Speichermedien**.
1. Mindestens **eine Kopie** befindet sich **an einem anderen physischen Standort**.
1. Mindestens **eine Kopie ist offline und unveränderlich**, z. B. durch Versionskontrolle.
0. **Null Fehler** in den Backups – alle Versionen werden regelmäßig überprüft.

Umsetzung in der IPA

Zur Sicherstellung der Datenintegrität und Nachvollziehbarkeit habe ich folgende Backup-Strategie umgesetzt:

1. **Lokale Speicherung:** Die aktuelle Version der Dokumentation wird lokal auf meinem Rechner unter Dokumente/IPA gesichert. (siehe Abbildung 5.1)

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo

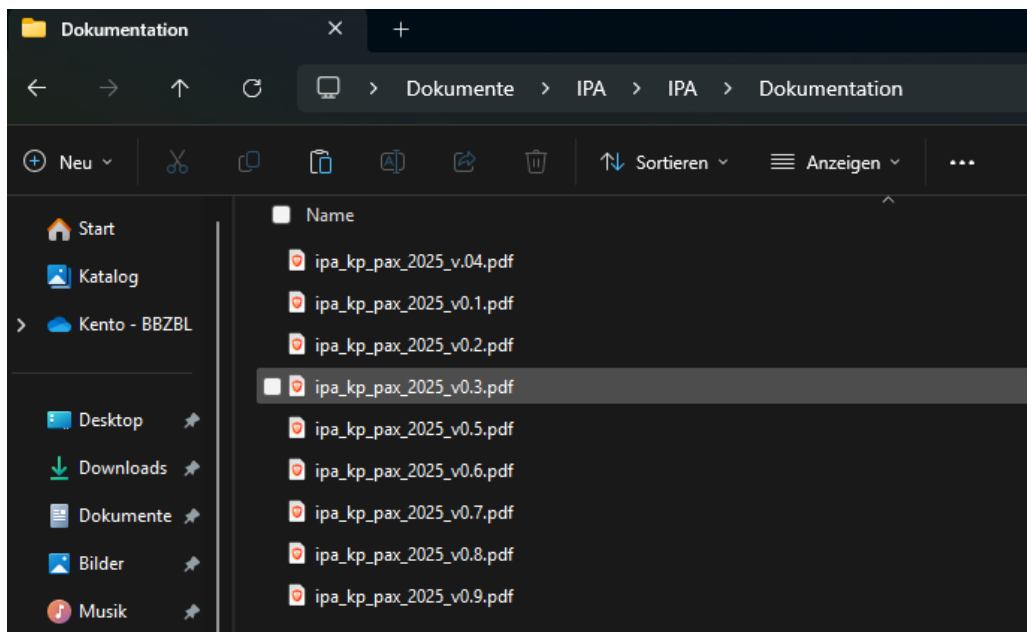


Abbildung 5.1: Lokale Sicherung im Verzeichnisstruktur „Dokumente/IPA“

2. **Cloud-Speicherung via Google Drive:** Täglich wird die aktuelle Version zusätzlich auf Google Drive hochgeladen. Jede Version wird dabei mit eindeutiger Versionsnummer gespeichert. (siehe Abbildung 5.2)



Abbildung 5.2: Tägliche Sicherungen auf Google Drive mit klarer Versionierung

3. **Versionskontrolle via GitHub:** Parallel dazu verwalte ich das Projekt in einem öffentlichen GitHub Repository. Durch Git-Commits werden Änderungen dokumentiert und nachvollziehbar archiviert. (siehe Abbildung 5.3)

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen

Kandidat: Kento Puleo

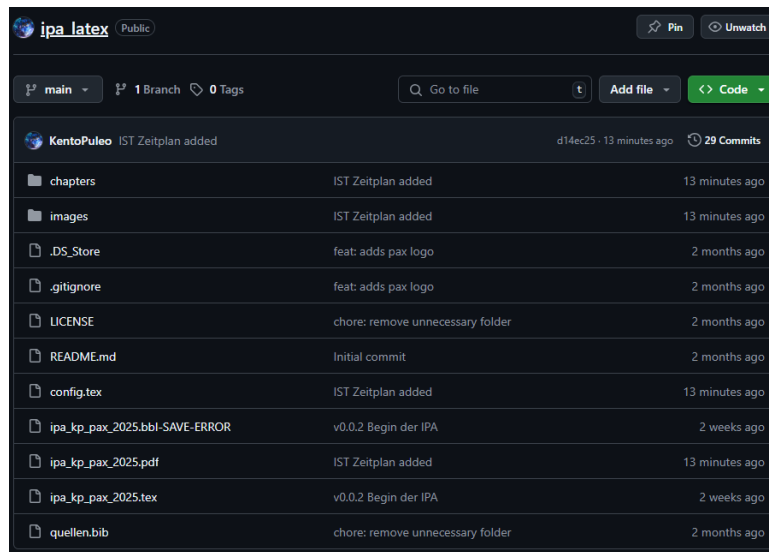


Abbildung 5.3: Versionierung und Backup auf GitHub mit Commit-Historie

Diese Strategie erfüllt die **3-2-1-1-0-Regel** wie folgt:

- **3 Kopien:** Lokal, Google Drive, GitHub.
- **2 Medientypen:** Lokale Festplatte (offline), Cloud-Speicher (Drive) und Versionsverwaltungssystem (Git).
- **1 geografisch unabhängiger Ort:** Google Drive liegt in der Cloud – physisch unabhängig vom eigenen Rechner.
- **1 Offline-Kopie:** Lokale Speicherung sowie nicht direkt bearbeitbare PDF-Versionen in Drive.
- **0 Fehler:** Durch tägliche Sicherung und PDF-Export werden fehlerhafte Zwischenstände minimiert.

Die tägliche Sicherung erfolgt manuell – durch Export und Upload jeder neuen Version. Änderungen sind im Git-Repository zusätzlich transparent dokumentiert.

KAPITEL 6

ZEITPLAN

Für das vorliegende IPA-Projekt wurde ein strukturierter Zeitplan erstellt, um die Durchführung systematisch zu planen und zu überwachen. Die folgende Version stellt die überarbeitete Fassung (v2) dar, welche auf Basis der ersten Planung angepasst wurde. Alle vorherigen Versionen des Zeitplans sind zur Nachvollziehbarkeit im Anhang dokumentiert.

Der Zeitplan ist in einzelne Arbeitspakete unterteilt, die sich jeweils über definierte Zeitabschnitte erstrecken. Die farblich grün hervorgehobenen Zeilen kennzeichnen den Beginn einer neuen Phase entsprechend der gewählten Projektmethode (Wasserfallmodell). Diese markieren zugleich die wichtigsten Meilensteine im Projektverlauf.

- Meilenstein 1 - Abschluss des Setups
- Meilenstein 2 - Abschluss der Analyse-Phase
- Meilenstein 3 - Anschluss der Design-Phase
- Meilenstein 4 - Abschluss der Implementierung
- Meilenstein 5 - Abschluss der Test/Qualitätssicherung Phase
- Meilenstein 6 - Abschluss der Ergebnisse und Ausblick Phase
- Abgabe

Auf den folgenden zwei Seiten werden die geplanten (SOLL) und tatsächlich umgesetzten (IST) Abläufe des Projekts gegenübergestellt. So lässt sich transparent nachvollziehen, inwieweit das Projekt im geplanten Rahmen geblieben ist oder Anpassungen notwendig wurden.

[illegible]

Abbildung 6.2: IST Zeitplan der Projektarbeit (Quelle: Eigenerarbeitung)

KAPITEL 7

PERSÖNLICHES FAZIT

Persönliche Schlussfolgerungen und Erkenntnisse.

KAPITEL 8

ARBEITSJOURNAL

Das Arbeitsjournal dient der Nachvollziehbarkeit der Arbeiten, welche Kento Puleo zwischen dem 06.05.2025 und dem 21.05.2025 getätigt hat. Ebenso dient es zur Reflexion.

Jedes Arbeitsjournal ist gleich aufgebaut. Zuerst werden alle Tätigkeiten aufgeschrieben und Probleme oder Erfolge vermerkt. In einem weiteren Text wird eine persönliche Reflexion der Arbeit wiedergegeben. Zum Schluss wird in einer Tabelle aufgeführt, welche Arbeitspakete an diesem Tag erledigt werden konnten.

Jedes Arbeitsjournal wird vom Kandidaten (Kento Puleo) und dem Berufsbildner (Sascha Listner) unterschrieben. Das Arbeitsjournal wird täglich geführt.

Das Arbeitsjournal ist im Anhang zu finden.

Teil II

Projektdokumentation

KAPITEL 9

KURZFASSUNG

Kurze Zusammenfassung der Arbeit.

9.1 Ausgangslage

Beschreibung der Ausgangssituation.

9.2 Zielsetzung

Beschreibung der zu erreichenden Ziele.

9.3 Ergebnis

Beschreibung der erzielten Ergebnisse.

KAPITEL 10

INITIALISIERUNG/ANALYSE

In diesem Kapitel wird die aktuelle Ausgangslage analysiert, Schwachstellen identifiziert sowie die angestrebten Ziele und Anforderungen beschrieben.

10.1 Erweiterung der Ausgangslage

10.1.1 IST-Stand

Benutzer, die sich über die EcoHub-Plattform bei Pax anmelden möchten, müssen aktuell initial manuell über eine interne Eingabemaske im LDAP-Verzeichnis angelegt werden. Dabei wird zwischen Private Vorsorge (PV) und Berufliche Vorsorge (BV) unterschieden. Ein Broker, der durch diese Maske im LDAP erfasst wird, erhält anschliessend die notwendigen Rollen und Gruppen.

Dieser Prozess erfolgt manuell, ist fehleranfällig und verursacht hohen administrativen Aufwand.

Sollte sich ein Broker anmelden wollen, der nicht im LDAP vorhanden ist, scheitert der Vorgang, da aktuell keine automatische Benutzererstellung existiert. Die vorhandene `addUser`-Funktion generiert eine IDP-Nummer, welche für die Durchführung des Single Sign-On (SSO) zwingend erforderlich ist. Die Web Access Management (WAM) prüft im Rahmen des SSO auf das Vorhandensein dieser IDP-Nummer. Fehlt sie, wird der Zugriff verweigert.

Auf der EcoHub-Plattform existieren bereits eigene Berechtigungskonzepte. Diese stimmen grösstenteils mit jenen der Pax überein. Es wird zwischen Einzelleben (EL) und Kollektivleben (KL) unterschieden (siehe Abbildung 10.1). Obwohl diese Rollen über EcoHub zugewiesen werden können, werden sie derzeit nicht automatisch in das LDAP-Verzeichnis übernommen und operativ nicht weiterverwendet.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Einzelleben (PV) (EL_*)

Diese Rollen beziehen sich auf **Einzelverträge** der Lebensversicherung (Privatkunden).

Rollenname	Code	Beschreibung
Bestandsdaten abrufen	EL_INFO	Zugriff auf Kundendaten zu Einzelverträgen (Details, Rendite etc.).
Offertwesen	EL_OFFERT	Berechtigt zur Nutzung des Offertrechners (z. B. Erstellung neuer Angebote).
Vertragsverwaltung	EL_VERTRAG	Berechtigt zur Anpassung von Kunden- und Vertragsdaten.
Leistungsfälle	EL_LEISTUNG	Einsicht in laufende Leistungsfälle (z. B. Todesfall-Leistungen).

Kollektivleben (BV) (KL_*)

Diese Berechtigungen betreffen **Gruppenverträge**, z. B. BVG (Pensionskassen).

Rollenname	Code	Beschreibung
Bestandsdaten abrufen	KL_INFO	Zugriff auf Kundendaten zu Gruppenverträgen.
Offertwesen	KL_OFFERT	Berechtigt zur Nutzung des Offertrechners für Firmenkunden.
Vertragsverwaltung	KL_VERTRAG	Berechtigt zur Anpassung von Kollektivverträgen.
Eigene BVG-Verträge	KL_EIGENVERTRAG	Einsicht in eigene BVG-Verträge (Pensionskassenverwaltung).

Abbildung 10.1: Rollenmodell auf der EcoHub-Plattform (Quelle: Eigenerarbeitung)

10.1.2 LDAP-Verzeichnisstruktur

Zur besseren Nachvollziehbarkeit der bestehenden Benutzerverwaltung wurde im Rahmen der Analysephase auch die aktuelle LDAP-Struktur untersucht. Abbildung 10.2 zeigt den lokalen Aufbau des LDAP-Verzeichnisses im Projektkontext.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo

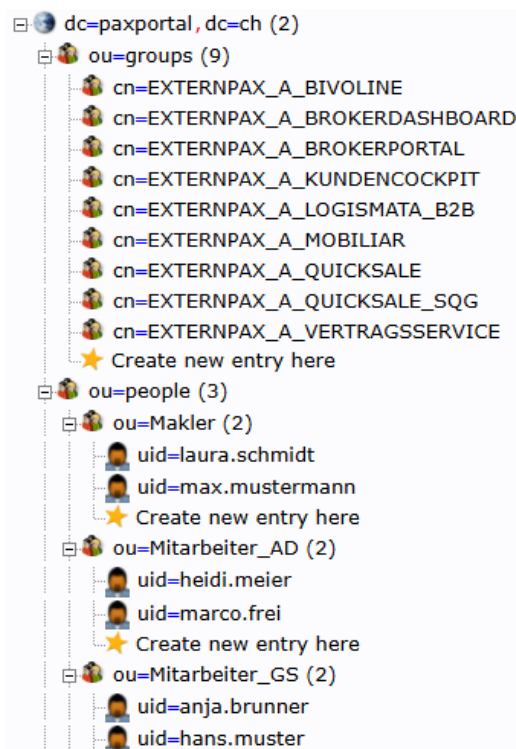


Abbildung 10.2: LDAP-Verzeichnisstruktur im lokalen Setup (Quelle: Eigenerarbeitung)

Die Struktur gliedert sich wie folgt:

- **Basis-Domain:** `dc=paxportal,dc=ch`
- **Organisatorische Einheiten (OUs):**
 - `ou=groups`: Enthält LDAP-Gruppen, welche externen Rollen zugewiesen sind. Die Gruppen folgen dem Namensschema `cn=EXTERNPAX_A_<ROLLE>` und bilden verschiedene Anwendungen wie `BROKERPORTAL`, `LOGISMATA_B2B` oder `QUICKSALE` ab.
 - `ou=people`: Beinhaltet Benutzerobjekte, unterteilt in:
 - * `ou=Makler`: für Broker-Nutzer
 - * `ou=Mitarbeiter_AD`: für interne AD-Mitarbeitende
 - * `ou=Mitarbeiter_GS`: für weitere interne Mitarbeitende

Die im Bild gezeigten Benutzereinträge (`uid=laura.schmidt`, `uid=hans.muster` etc.) sind **ausschliesslich Dummy-Daten** und dienen lediglich der Veranschaulichung.

Relevanz für das Projekt

Diese LDAP-Struktur ist zentral für den *EcoHub Provisioning Service*, welcher Benutzer automatisiert erstellt, aktualisiert oder löscht. Dabei ist besonders die rollenbasierte Gruppenzuweisung entscheidend: Über ein

externes Mapping (z. B. EL_, KL_) wird gesteuert, in welche Gruppen ein Benutzer aufgenommen wird.

Datenschutz und Veröffentlichung

Vor der Integration dieser Struktur in das GitHub-Repository wurde geprüft, ob die cn-Bezeichner der Gruppen bedenkenlos veröffentlicht werden dürfen. Diese Klärung war notwendig, um sicherzustellen, dass keine vertraulichen oder geschützten Produktionsinformationen offengelegt werden. Die hier gezeigte Struktur ist vollständig anonymisiert und enthält keine personenbezogenen oder produktiven Daten.

10.1.3 Business Process Model der updateUser-Funktion (BPM)

Eine *updateUser*-Funktion ist vorhanden, erfüllt jedoch noch nicht die gewünschten Anforderungen hinsichtlich automatischer Rechtezuweisung oder Synchronisierung. Wie in der Abbildung 10.3 zu sehen ist, wird bei der updateUser-Funktion die gleiche Logik wie bei der addUser-Funktion verwendet.

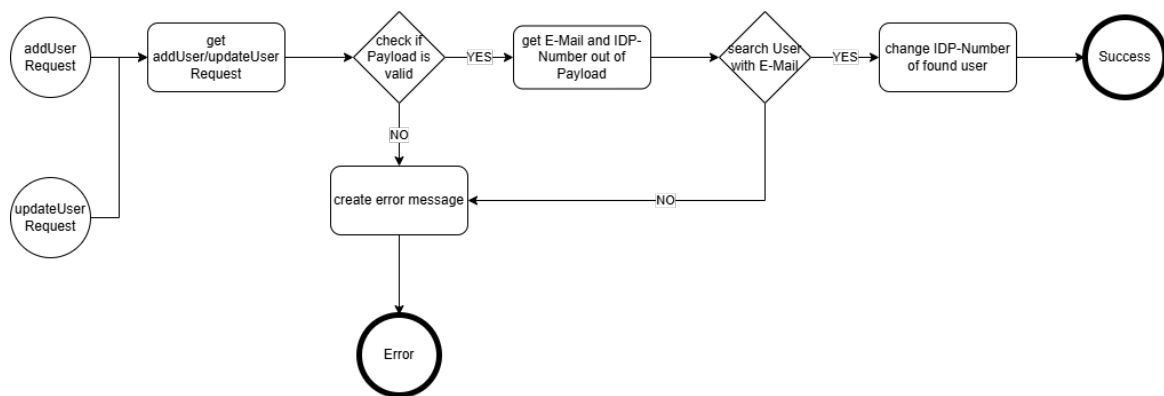


Abbildung 10.3: Business Process Model des IST-Zustands der Benutzerregistrierung (Quelle: Eigenerarbeitung)

10.1.4 Tabelle der zu testenden Endpunkte

Die folgende Tabelle gibt einen Überblick über die zu testenden Endpunkte und beschreibt deren aktuelle Funktionalitäten.

Tabelle 10.1: Übersicht der zu testenden Endpunkte

Endpoint	Beschreibung
addUser	Sucht einen Broker im LDAP-Verzeichnis anhand der E-Mail-Adresse und weist diesem eine IDP-Nummer zu.
updateUser	Aktualisiert die IDP-Nummer im LDAP-Verzeichnis.
deleteUser	Entfernt die IDP-Nummer eines Benutzers aus dem LDAP-Verzeichnis.

Fortsetzung von Tabelle 10.1

Endpoint	Beschreibung
getUserStatus	Prüft die Existenz der IDP-Nummer eines Benutzers im LDAP-Verzeichnis.

10.1.5 Struktur der SOAP-Nachrichten

Die Benutzerprovisionierung erfolgt über eine SOAP-Schnittstelle, bei der unterschiedliche Endpunkte wie `addUser`, `updateUser`, `deleteUser` und `getUserStatus` verwendet werden. Die meisten Endpunkte besitzen eine eigene XML-Struktur, welche den jeweiligen Verwendungszweck widerspiegelt. Die Struktur ist vorgegeben, wurde aber im Rahmen der Analyse hinsichtlich Relevanz und Vollständigkeit geprüft.

Endpunkt: `addUser`

Die folgende Nachricht zeigt eine typische `addUser`-Anfrage. Die wichtigsten übermittelten Informationen betreffen die Benutzeridentifikation (`idpUserId`), Rollenzuweisung (`igRoles`) sowie persönliche Angaben.

Listing 10.1: Beispielhafte `addUser`-Anfrage

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>123</idpUserId>
      <orgId>1</orgId>
      <igRoles>EL_OFFERT</igRoles>
      <orgRegNo>10</orgRegNo>
      <primaryOU>
        <id>1</id>
        <regNo>10</regNo>
        <type>test</type>
        <name>test</name>
      </primaryOU>
      <employed>true</employed>
      <givenName>max</givenName>
      <surname>mustermann</surname>
      <userEmail>max.mustermann1@email.ch</userEmail>
      <userOfficePhone>123</userOfficePhone>
      <userMobilePhone>123</userMobilePhone>
      <userBirthday>1970-01-01T00:00:00Z</userBirthday>
      <userSex>M</userSex>
      <userLanguage>DE</userLanguage>
      <validFrom>2024-01-01T00:00:00Z</validFrom>
      <validTo>2024-12-31T00:00:00Z</validTo>
      <userType>PERSONAL</userType>
      <userCategory>BROKER</userCategory>
      <creationDate>2024-01-01T00:00:00Z</creationDate>
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<creationUser>admin</creationUser>
<modificationDate>2024-01-01T00:00:00Z</modificationDate>
<modificationUser>admin</modificationUser>
<requestId>1</requestId>
<requestTime>2024-01-01T00:00:10Z</requestTime>
</user:addUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Endpunkt: updateUser

Bei der updateUser-Anfrage wird die gleiche XML-Struktur verwendet wie bei der addUser-Anfrage.

Endpunkt: deleteUser

Die folgende Nachricht zeigt eine typische deleteUser-Anfrage. Die wichtigsten übermittelten Informationen betreffen die Benutzeridentifikation (idpUserId).

Listing 10.2: Beispielhafte addUser-Anfrage

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
userProvisioningSchemaTypes-v3.0/">
<soapenv:Header/>
<soapenv:Body>
  <user:deleteUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
    <idpUserId>222</idpUserId>
    <deletionDate>2024-12-01T00:00:00Z</deletionDate>
    <deletionUser>admin</deletionUser>
    <requestId>1</requestId>
    <requestTime>2024-12-01T00:00:10Z</requestTime>
  </user:deleteUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Endpunkt: getUserStatus

Die folgende Nachricht zeigt eine typische getUserStatus-Anfrage.

Listing 10.3: Fehlermeldung bei addUser-Anfrage

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Processing add user failed for idpUserId=123</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Antwortstruktur bei Fehlern

Im Erfolgsfall erfolgt keine Rückmeldung. Tritt jedoch ein Fehler auf, wird eine Antwort mit einem Fehlercode und einer Nachricht geliefert, folgend ein Beispiel:

Listing 10.4: Fehlermeldung bei addUser-Anfrage

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnTypes xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Processing add user failed for idpUserId=123</errorMessage>
    </ns3:commonReturnTypes>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Feldrelevanz

Folgende Felder sind für die weitere Verarbeitung im Provisioning-Service besonders relevant:

- **idpUserId:** Eindeutige Identifikation des Benutzers im LDAP
- **igRoles:** Grundlage für rollenbasierte Gruppenzuweisung
- **userEmail:** Wird als Mail-Attribut in LDAP übernommen
- **requestTime:** Technisches Feld zur Nachvollziehbarkeit

Die XML-Struktur bildet somit das zentrale Bindeglied zwischen dem externen Aufrufer und der internen LDAP-Provisionierung.

10.1.6 Analyse der bestehenden CI/CD-Pipeline

Zur Qualitätssicherung und Automatisierung der Build- und Testprozesse wird im Projekt eine GitHub-Actions-basierte CI/CD-Pipeline eingesetzt. Diese ist in der Datei `pipeline.yml` definiert und wird bei jedem push ins Repository ausgeführt.

Übersicht der Jobs

Die Pipeline besteht aus vier zentralen Jobs:

- **test:** Führt automatisierte Unit-Tests mit Maven aus
- **build:** Erstellt das Projekt und bereitet ein Docker-Image vor
- **publish:** Veröffentlicht das Docker-Image in eine Container Registry (nur bei Tags)
- **delete_artifacts:** Entfernt erstellte Artefakte nach erfolgreicher Veröffentlichung

Technische Details (Auszug Job test)

Listing 10.5: Auszug aus pipeline.yml

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      -uses: actions/checkout@v4
      -name: Set up JDK
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
      -name: Execute tests
        run: chmod +x ./mvnw && ./mvnw test
```

Die Tests werden vor dem Build ausgeführt und dienen als Quality-Gate. Nur bei erfolgreichem Testdurchlauf wird das Projekt gebaut und ein Docker-Image erzeugt.

Integration in den Entwicklungsprozess

Diese Pipeline bildet die Grundlage für die kontinuierliche Validierung und Bereitstellung des Provisioning-Dienstes. Dadurch wird sichergestellt, dass:

- Änderungen stets mit bestehenden Funktionalitäten kompatibel sind
- Fehler frühzeitig erkannt werden
- Releases nur bei validierten Git-Tags veröffentlicht werden

10.1.7 Darstellung der Systemstruktur im IST-Stand

Folgend wird die Systemstruktur im IST-Stand dargestellt. Diese wird während dieser Arbeit jedoch nicht verändert und dient nur zur Veranschaulichung des aktuellen Zustands.

Systemkontextdiagramm (C1)

Zur Veranschaulichung des IST-Zustands wurde ein C1-Diagramm nach dem C4-Modell erstellt. Dieses stellt die Systemlandschaft im Kontext dar und zeigt die Interaktionen zwischen Benutzer, zentralem Dienst (Eco-Hub), dem Provisioning Service sowie den Umsystemen.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo

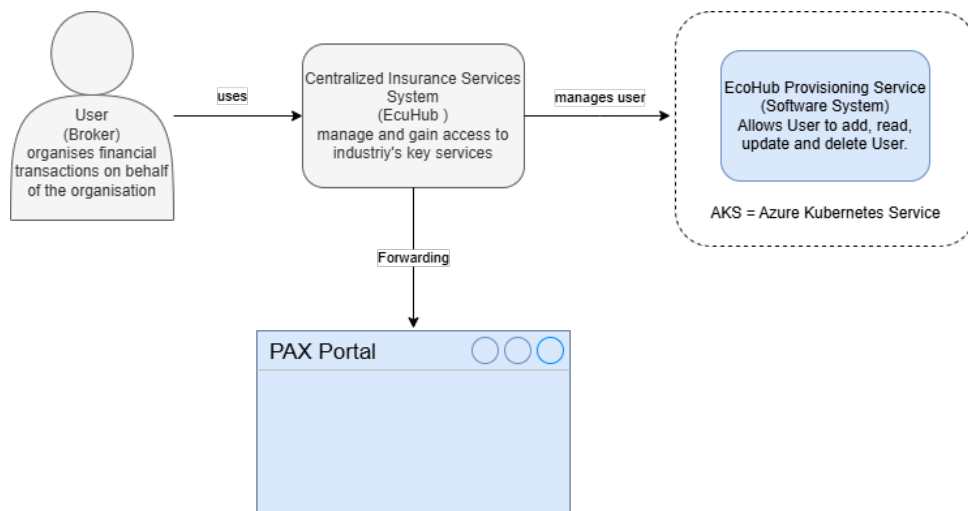


Abbildung 10.4: Systemkontext (C1) - IST-Zustand der Benutzerverwaltung (Quelle: Eigenerarbeitung)

Container-Diagramm (C2)

Ergänzend zum Systemkontext (C1) wird in der folgenden Abbildung die interne Struktur des Systems im Sinne eines C2-Diagramms dargestellt. Dieses zeigt die einzelnen technischen Container, deren Hauptaufgaben sowie die Kommunikationsflüsse zwischen ihnen im aktuellen Systemzustand.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen

Kandidat: Kento Puleo

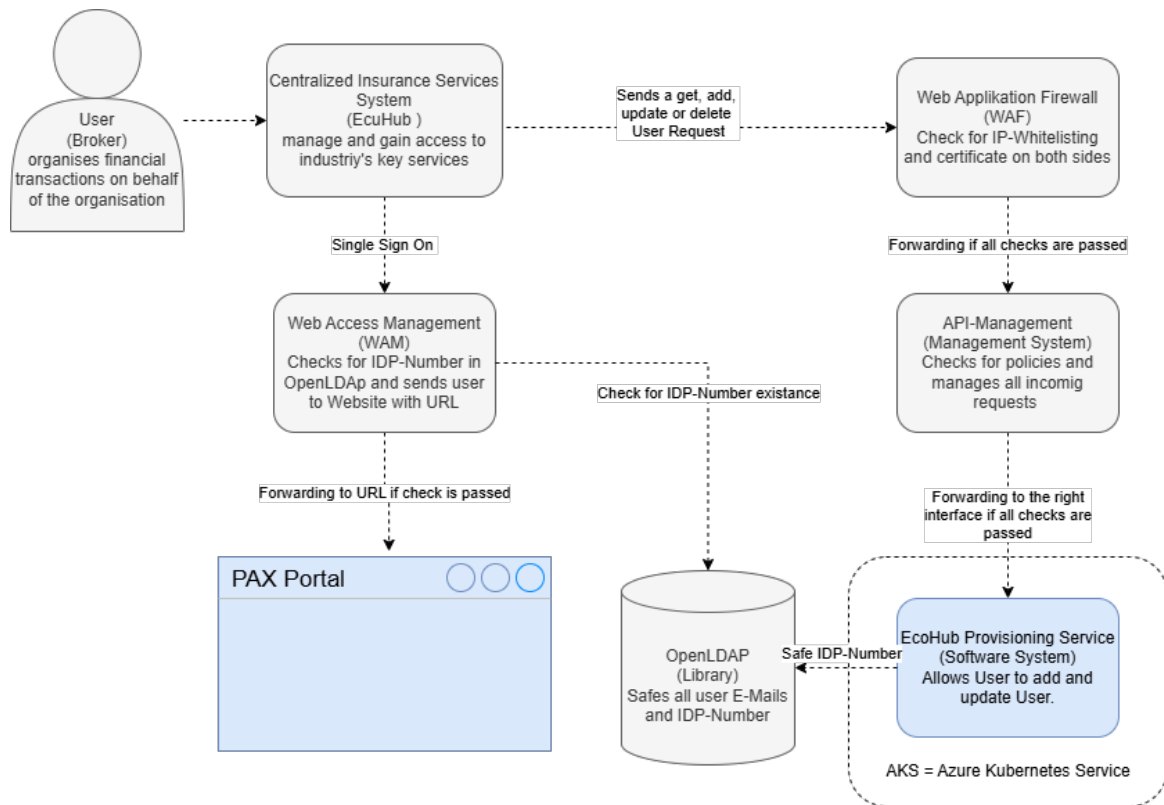


Abbildung 10.5: C2-Diagramm IST-Zustand - Containerübersicht (Quelle: Eigenerarbeitung)

Die wichtigsten Container im aktuellen System sind:

- **Web Application Firewall (WAF):** Führt IP-Whitelisting sowie Zertifikatsprüfungen durch und schützt die Schnittstellen gegenüber dem Internet.
- **API-Management:** Verwaltet eingehende API-Anfragen, prüft auf Richtlinien (Policies) und leitet validierte Anfragen an die zuständigen Systeme weiter.
- **Web Access Management (WAM):** Übernimmt die zentrale Authentifizierung (SSO) und prüft anhand der IDP-Nummer im LDAP, ob ein Benutzer autorisiert ist.
- **OpenLDAP:** Verzeichnisdienst zur Speicherung von Benutzerdaten, insbesondere E-Mail-Adressen und IDP-Nummern.
- **EcoHub Provisioning Service:** Eine Spring Boot-Anwendung, bereitgestellt auf Azure Kubernetes (AKS), welche Benutzer über SOAP-Schnittstellen verwalten kann (hinzufügen, aktualisieren, löschen).
- **PAX Portal:** Die Zielplattform, auf die Broker nach erfolgreichem Login weitergeleitet werden, wenn alle Prüfungen bestanden wurden.

Die Kommunikation zwischen diesen Containern erfolgt aktuell sequenziell, mit mehreren Prüfungen und Sicherheitsbarrieren. Änderungen oder Fehler in einem der Container können sich direkt auf das Verhalten des Gesamtsystems auswirken.

Komponentendiagramm (C3) - EcoHub Provisioning Service

Das nachfolgende C3-Diagramm zeigt den internen Aufbau des **EcoHub Provisioning Service** auf Komponentenebene. Der Service ist eine Spring Boot-Anwendung. Er stellt eine SOAP-Schnittstelle zur Verwaltung von Broker-Benutzern bereit. Die Benutzerverwaltung erfolgt durch Interaktion mit einem extern angebundenen OpenLDAP-Verzeichnisdienst.

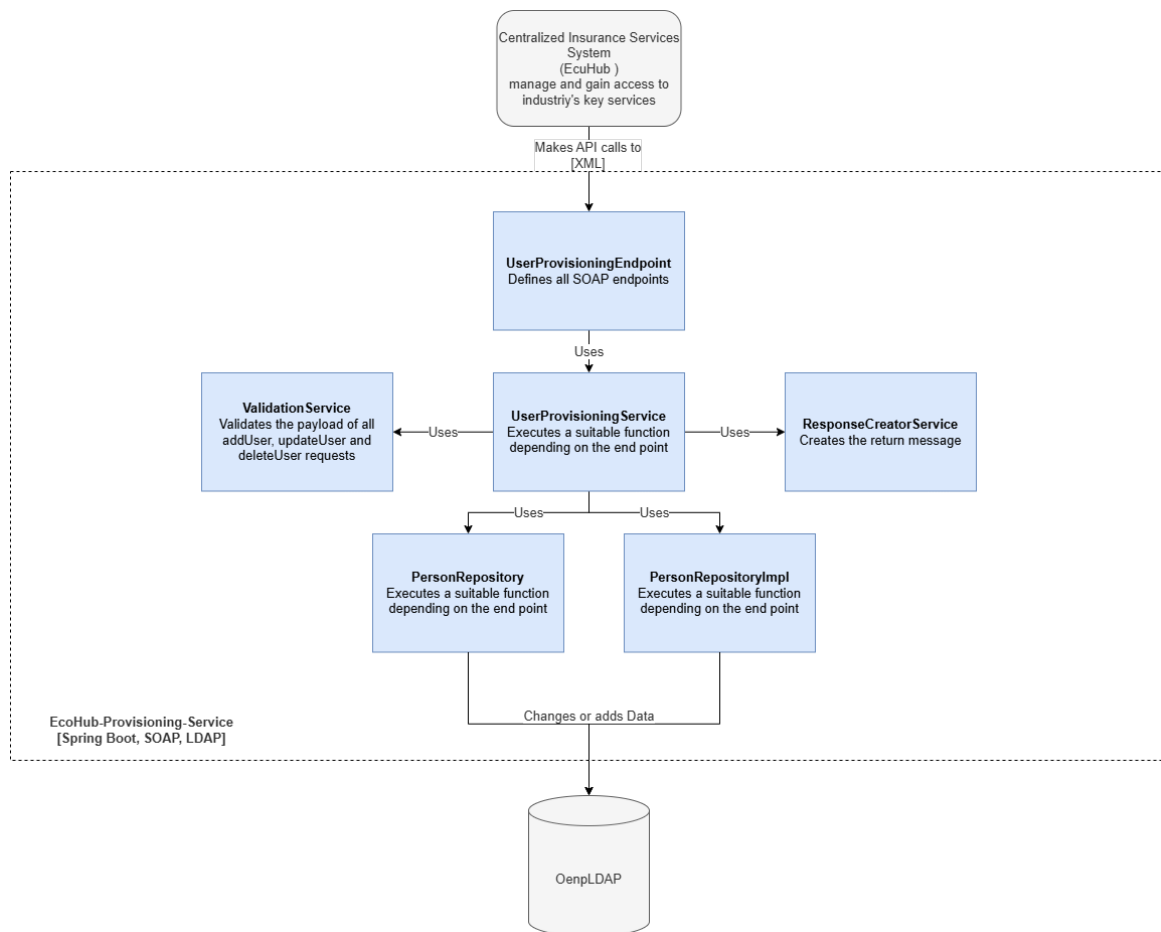


Abbildung 10.6: C3-Komponentendiagramm - EcoHub Provisioning Service (Quelle: Eigenerarbeitung)

Die wichtigsten Komponenten im Diagramm sind:

- **UserProvisioningEndpoint**
SOAP-Webservice-Endpunkt, der externe Anfragen wie `addUser`, `updateUser`, `deleteUser` oder `getUserStatus` entgegennimmt.
- **UserProvisioningService**
Zentrale Geschäftslogik. Validiert die Anfragen, delegiert LDAP-Operationen und erstellt Rückgabemeldungen.

- **ValidationService**

Führt strukturierte Prüfungen der eingehenden Payloads durch (z. B. Pflichtfelder, Formate, Längenbeschränkungen) und verwendet Adapter-Objekte zur Entkopplung vom SOAP-Modell.

- **ResponseCreatorService**

Baut standardisierte Antwortobjekte (inkl. Fehlermeldungen) zur Rückgabe über SOAP.

- **PersonRepository & PersonRepositoryImpl**

Repositories zur Suche, Änderung und Löschung von Nutzerdaten im OpenLDAP-Verzeichnis. Nutzen `LdapTemplate` von Spring.

- **OpenLDAP (extern)**

Verzeichnisdienst, der Brokerdaten und ihre Rollen speichert. Wird durch den Provisioning Service gelesen und beschrieben.

- **EcoHub (extern)**

Die EcoHub-Plattform ruft den Provisioning-Service über eine definierte WSDL-Schnittstelle auf und sendet XML-basierte Requests.

Die Kommunikation erfolgt synchron via SOAP, die Daten werden direkt im LDAP-Verzeichnis persistiert. Alle Komponenten sind lose gekoppelt und durch Interfaces und Adapter abstrahiert, was Testbarkeit und Erweiterbarkeit verbessert.

10.1.8 Problemanalyse

Schwachstellen sind konkrete Mängel oder Probleme im aktuellen System oder Prozess. Sie beschreiben, was heute nicht gut funktioniert und somit den Änderungsbedarf begründet. Darunter fallen aktuell folgende Punkte:

- Der Benutzer muss manuell im LDAP eingetragen werden, was ein hoher Aufwand ist und die manuelle Bearbeitung birgt ein hohes Fehlerrisiko.
- Die Rollen von EcoHub werden nicht im LDAP übernommen, was bedeuten kann, dass Broker im EcoHub andere Berechtigungen haben.
- Es gibt keine Tests auf die Endpunkte des EcoHub-Services. Schlägt ein Endpunkt fehl oder werden Änderungen gemacht, wird der Nutzer es als erstes bemerken.

10.1.9 Chancen und Risiken

Risiken sind mögliche Probleme oder Gefahren, die beim Projektverlauf oder der Umsetzung der Änderung auftreten könnten - also zukünftige Unsicherheiten. Folgende wurden für dieses Projekt erkannt.

- Fehlerhafte Zuordnung von Rollen. Die Mapping-Funktion weist den Broker in falsche Gruppen und Rollen.

- Änderung an LDAP-Struktur. Ungründliche Vorbereitung und Arbeiten könnte die LDAP-Struktur verändern.
- Unvollständige Testabdeckung. Werden die Funktionen oder Endpunkte nicht vollständig abgedeckt, kann es weiterhin zu unentdeckten Problemen führen.

10.1.10 Zielsetzung

Nun sind verschiedene Ziele für das EcoHub vorgesehen. Der ganze Prozess soll automatisiert geschehen, das bedeutet die manuelle Erstellung der Broker über die Maske fällt weg. Der Broker soll die Möglichkeit bekommen, über das EcoHub sich bei der Pax anzumelden, insofern er eine Service-Vereinbarung eingegangen ist, und damit ein neuer Nutzer im LDAP erstellt und gespeichert wird. Bei der Erstellung wie aber auch nachträglich sollen die Berechtigungen, die der Broker auf der EcoHub-Plattform erhält, die gleichen sein wie die im LDAP. Auch die einzelnen Schnittstellen des EcoHub-Services sollen getestet werden, um Probleme vorzeitig zu erkennen und zu lösen.

10.1.11 Abgrenzung

Wichtig ist, sich nicht in der eigenen Arbeit zu verlieren oder sich in andere Aufgaben zu verlieren. Die Automatisierung des Prozesses, um einen neuen Broker hinzuzufügen, ist nicht Teil der IPA. Auch dass die Gruppen-Zuteilung bei der initialen Anmeldung geschieht, ist nicht Teil der IPA.

10.2 Erweiterte Aufgabenstellung

Die Aufgabe in dieser IPA bezieht sich auf zwei der Ziele. Es soll eine Mapping-Funktion geschrieben werden welche die EcoHub Rollen mit den internen Rollen verbindet. Das soll geschehen sollten sich die Rollen des Broker ändern und ein update nötig sein. Auch sollen alle Schnittstellen des EcoHub-Services automatisiert getestet werden, die Automation soll über eine Pipeline laufen.

10.3 Anforderungen

10.3.1 Funktionale Anforderungen

Bei Funktionalen Anforderungen handelt es sich um das (WAS?). Was soll das System oder Produkt tun, besser gesagt welche Funktionen oder Verhaltensweisen muss es bieten. Folgend die wichtigsten Funktionalen Anforderung im bezug auf die zwei Aufträge.

Mapping-Funktion

- Der Broker wird entsprechend seiner EcoHub-Rolle den korrekten Gruppen im LDAP zugeordnet.

- Gruppen, in denen der Broker nicht mehr Bestandteil ist, aus denen wird er entfernt.
- Bei nicht bekannten Rollen wird die Anfrage ignoriert und der Broker kriegt keine neuen Berechtigungen.
- Die Funktion muss das Mapping aus einer externen Konfigurationsdatei (z.B. application.yml) laden, damit Änderungen ohne Codeanpassung möglich sind.
- Die Funktion muss mehrere Rollen gleichzeitig verarbeiten können.

API-Testautomatisierung

- Alle bekannten API-Endpunkte des EcoHub-Provisioning-Services müssen durch automatisierte Tests abgedeckt werden.
- Die Tests müssen sicherstellen das konkrete Fehlermeldungen zurückgegeben werden.
- Tests müssen jederzeit wiederholbar sein, ohne manuelle Vorbereitung (z.B. Setup- und Teardown-Routinen für Testdaten).
- Die Tests müssen nicht nur wiederholbar sondern auch automatisch durchgeführt werden.

10.3.2 Nicht-funktionale Anforderungen

Bei nicht-funktionalen Anforderungen handelt es sich um das (WIE?). Wie soll das System funktionieren und welche Qualität soll dieses bieten.

Mapping-Funktion

- Die Verarbeitung einer Mapping-Anfrage soll maximal 500 ms dauern.
- Ungültige oder unbekannte Rollen verursachen einen kontrollierten Fehler mit Logging.
- Die Mapping-Logik muss durch Unit-Tests abdeckbar sein.

API-Testautomatisierung

- Die komplette Testausführung soll in unter 2 Minuten durchführbar sein, um schnelle Feedback-Zyklen zu ermöglichen.
- Für die Tests dürfen keine sensiblen Daten verwendet werden. Es werden Testdaten genutzt, welche nicht mit echten Nutzern in Verbindung gebracht werden können.
- Die Testfälle müssen gut strukturiert, benannt und dokumentiert sein, sodass sie auch von Dritten gepflegt werden können.
- Das Testsystem soll erweiterbar sein, damit neue Endpunkte mit geringem Aufwand integriert werden können.

10.4 Rahmenbedingungen und Annahmen

Rahmenbedingungen beschreiben äussere Faktoren und definieren den Spielraum, der das Projekt nicht aktiv beeinflusst, an den sich aber gehalten werden muss. Für diese Arbeit gelten folgende Rahmenbedingungen:

1. Das bestehende LDAP-System (OpenLDAP) darf nicht strukturell verändert werden (Schema, Gruppenstruktur bleibt gleich).
2. Es muss eine bestehende Spring Boot Applikation verwendet werden.
3. Es dürfen nur bereits vorhandene Gruppen im LDAP verwendet werden.
4. Die Integration erfolgt via SOAP-Schnittstelle des EcoHub.
5. Die API-Tests werden mit Playwright gemacht, da Pax weit alle API- und End-to-End-Tests mit Playwright geschrieben werden.

KAPITEL 11

PLANUNG/DESIGN

Gemäss dem Wasserfallmodell folgt auf die Analysephase die Designphase, in der die geplanten Systemanpassungen detailliert ausgearbeitet werden. Aufbauend auf den zuvor identifizierten Schwächen wird in diesem Kapitel beschrieben, wie die Benutzerverwaltung des EcoHub Provisioning Service funktional erweitert und technisch abgesichert werden soll.

Neben der Erweiterung bestehender Funktionalität steht auch die Einführung automatisierter Tests zur Qualitätssicherung im Fokus. Die folgenden Abschnitte stellen das Zielbild, die geplanten Änderungen auf Komponentenebene sowie die vorgesehenen Massnahmen zur Absicherung und Testbarkeit der Erweiterung vor.

11.1 Zielbild und Anforderungen

Im Rahmen der zweiten Projektphase werden zwei zentrale Erweiterungen am EcoHub Provisioning Service vorgenommen:

1. Die Funktion `updateUser` wird um eine automatische Gruppenzuweisung im LDAP erweitert. Die Zuweisung basiert auf den übermittelten Rolleninformationen (`igRoles`) und erfolgt konfigurierbar über ein YAML-basiertes Mapping.
2. Für alle vier vorhandenen SOAP-Endpunkte (`addUser`, `updateUser`, `deleteUser`, `getUserStatus`) werden API-Tests mit Playwright implementiert. Diese Tests werden in die bestehende CI/CD-Pipeline integriert, um eine frühzeitige Fehlererkennung zu ermöglichen.

Die Erweiterungen verfolgen das Ziel, die Benutzerverwaltung zu automatisieren und gleichzeitig die Qualitätssicherung durch systematische Tests zu erhöhen. Dabei wird die bestehende Architektur bewusst nur minimal angepasst und auf Wiederverwendbarkeit sowie Konfigurierbarkeit geachtet.

11.2 Komponentendesign: Erweiterung von updateUser

Die überarbeitete Version der updateUser-Funktion wurde im Rahmen der Design-Phase neu konzipiert und in einem BPMN-ähnlichen Ablaufdiagramm visualisiert (siehe Abbildung 11.1). Im Vergleich zum bisherigen Zustand wurde der Ablauf erweitert und strukturiert automatisiert. Neu wird der Benutzer aus allen bestehenden Gruppen entfernt, um eine fehlerfreie Neuzuordnung zu ermöglichen.

Die zentrale Neuerung besteht in der rollenbasierten Gruppenzuweisung: Anhand der im Payload enthaltenen igRoles und eines in der Konfiguration definierten Mappings wird der Benutzer automatisch den korrekten Gruppen (z.B. BROKERPORTAL, VERTRAGSSERVICE) hinzugefügt. Abschliessend wird die IDP-Nummer im LDAP aktualisiert. Fehlerhafte oder unvollständige Anfragen führen in einen klar definierten Fehlerpfad mit entsprechender Fehlermeldung.

Die neue Struktur ist robuster, besser wartbar und reduziert manuelle Eingriffe erheblich.

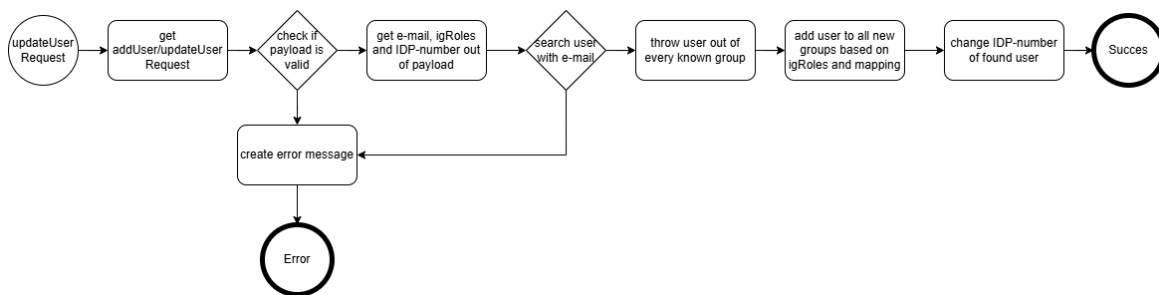


Abbildung 11.1: BPMN-Diagramm der updateUser-Funktion (Quelle: Eigenerarbeitung)

11.2.1 Konfigurationsdesign: YAML-Mapping

Um eine zentrale und flexible Steuerung der Gruppenzuweisung im LDAP-Verzeichnis zu ermöglichen, wird ein rollenbasiertes Mapping über die Konfigurationsdatei `application.yml` realisiert. Diese Konfiguration erlaubt es, bestimmte Rollenpräfixe, wie sie im SOAP-Payload unter `igRoles` übermittelt werden, vordefinierten Gruppen im LDAP-Verzeichnis zuzuordnen.

Die Gruppen befinden sich unter dem in der Konfiguration definierten `groupSearchBase ou=groups, dc=paxportal,dc=ch`. Für jede Rollenfamilie (z. B. `EL_` für Einzelleben oder `KL_` für Kollektivleben) ist eine Liste von Gruppen hinterlegt, in die Benutzer automatisch aufgenommen werden, sofern die entsprechende Rolle im Payload enthalten ist.

Die Abkürzungen EL und KL orientieren sich an der fachlichen Struktur der Pax:

- **EL** steht für *Einzelleben* und entspricht der *Privaten Vorsorge (PV)*.
- **KL** steht für *Kollektivleben* und entspricht der *Beruflichen Vorsorge (BV)*.

Die farbliche Markierung der Gruppen in Abbildung 11.2 veranschaulicht die Zuordnung:

- Gelb markierte Gruppen gehören zur Kategorie **EL**.

- Rot markierte Gruppen gehören zur Kategorie **KL**.

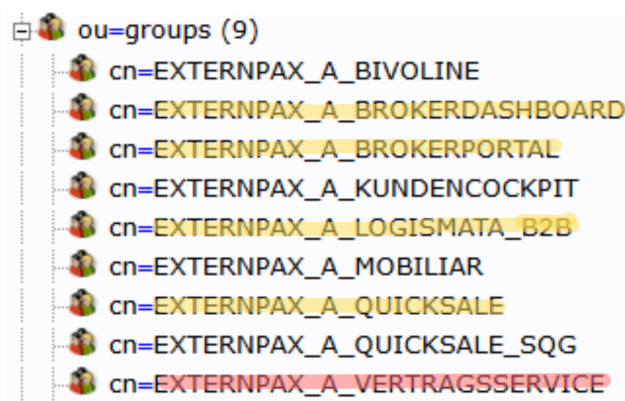


Abbildung 11.2: LDAP-Gruppen mit farblicher Zuordnung zu EL (gelb) und KL (rot) (Quelle: Eigenerarbeitung)

Nachfolgend ein exemplarischer Auszug aus der Konfiguration:

Listing 11.1: Auszug aus der YAML-Konfiguration für rollenbasiertes Mapping

```
ecohub:
  ldap:
    groupSearchBase: ou=groups,dc=paxportal,dc=ch
    roleMappings:
      EL_:
        - cn=EXTERNPAX_A_BROKERPORTAL,ou=groups
        - cn=EXTERNPAX_A_QUICKSALE,ou=groups
        - cn=EXTERNPAX_A_LOGISMATA_B2B,ou=groups
        - cn=EXTERNPAX_A_BROKERDASHBOARD,ou=groups
      KL_:
        - cn=EXTERNPAX_A_VERTRAGSSERVICE,ou=groups
```

Alle weiteren Gruppen, die in der Darstellung ersichtlich sind, aber nicht in der YAML-Konfiguration aufgeführt werden, stehen aktuell in keinem direkten Zusammenhang mit den für dieses Projekt relevanten Applikationen und wurden daher im Rahmen des Designs nicht berücksichtigt. Das Mapping kann jedoch jederzeit erweitert werden, sollte sich dies in Zukunft ändern.

11.3 Testdesign und API-Abdeckung

Zur Sicherstellung der Funktionalität und zur frühzeitigen Fehlererkennung werden für alle vier vorhandenen SOAP-Endpunkte systematische API-Tests eingeführt. Die Tests werden mit dem Framework **Playwright** entwickelt, welches durch seine Flexibilität sowohl für Web- als auch für API-Tests geeignet ist.

Zielsetzung

- Automatisierte Tests für: addUser, updateUser, deleteUser, getUserStatus
- Prüfung auf korrekte Verarbeitung valider Payloads
- Erkennung von Fehlerfällen bei ungültigen oder unvollständigen Eingaben
- Integration in die bestehende GitHub-Actions-Pipeline

11.4 Testfälle

Die Tests werden im gleichen Repository wie die Anwendung selbst gehalten. Es wird ein separater Ordner für die Tests erstellt. In der CI/CD-Pipeline wird ein dedizierter Job integriert, der die Tests bei jedem Build ausführt. So können fehlerhafte Implementierungen frühzeitig erkannt und rückgemeldet werden.

11.4.1 Vollständige XML-Payloads zu addUser, updateUser

Die Testfälle zeigen nur die wichtigsten Daten der Anfragen. Folgend daher die einmal die vollständigen XML-Payloads für die zwei Endpunkte.

Listing 11.2: addUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>-999</idpUserId>
      <orgId>1</orgId>
      <igRoles>EL_OFFERT</igRoles>
      <orgRegNo>10</orgRegNo>
      <primaryOU>
        <id>1</id>
        <regNo>10</regNo>
        <type>test</type>
        <name>test</name>
      </primaryOU>
      <employed>true</employed>
      <givenName>Marco</givenName>
      <surname>Frei</surname>
      <userEmail>marco.frei@pax.ch</userEmail>
      <userOfficePhone>123</userOfficePhone>
      <userMobilePhone>123</userMobilePhone>
      <userBirthday>1970-01-01T00:00:00Z</userBirthday>
      <userSex>M</userSex>
      <userLanguage>DE</userLanguage>
      <validFrom>2024-01-01T00:00:00Z</validFrom>
      <validTo>2024-12-31T00:00:00Z</validTo>
      <userType>PERSONAL</userType>
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```


Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen

Kandidat: Kento Puleo



```
<userCategory>BROKER</userCategory>
<creationDate>2024-01-01T00:00:00Z</creationDate>
<creationUser>admin</creationUser>
<modificationDate>2024-01-01T00:00:00Z</modificationDate>
<modificationUser>admin</modificationUser>
<requestId>1</requestId>
<requestTime>2024-01-01T00:00:10Z</requestTime>
  </user:addUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Nun die Testfälle zu den einzelnen Endpunkten:

11.4.2 Endpoint: addUser

Test-ID: 1

Beschreibung:

addUser - Nutzer erhält erfolgreich eine IDP-Nummer.

Input:

Listing 11.3: addUser-Anfrage - Erfolgreiche Registrierung

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
  userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType>
      <idpUserId>100</idpUserId>
      <givenName>Marco</givenName>
      <surname>Frei</surname>
      <userEmail>marco.frei@pax.ch</userEmail>
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.4: addUser - Erfolgreiche Antwort

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonResponseType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
    </ns3:commonResponseType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 2

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Beschreibung:

addUser - Nutzer mit dieser IDP-Nummer existiert bereits.

Input:

Listing 11.5: addUser-Anfrage - IDP-Nummer bereits vergeben

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType>
      <idpUserId>101</idpUserId>
      <givenName>Max</givenName>
      <surname>Mustermann</surname>
      <userEmail>max.mustermann@pax.ch</userEmail>
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.6: addUser - Nutzer existiert bereits

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_EXISTS</returnCode>
      <errorMessage>User already exists with idpUserId=101</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 3

Beschreibung:

addUser - Payload ist unvollständig (z.B. fehlt Nachname).

Input:

Listing 11.7: addUser-Anfrage - Fehlender Nachname

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType>
      <idpUserId>103</idpUserId>
      <givenName>Marco</givenName>
      <userEmail>marco.frei@pax.ch</userEmail>
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Erwartet:

Listing 11.8: addUser - Ungültiger Payload

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Payload validation failed for idpUserId=103</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 4

Beschreibung:

addUser - E-Mail-Adresse kann nicht verarbeitet werden (z.B. existiert nicht im internen System).

Input:

Listing 11.9: addUser-Anfrage - Falsche E-Mail

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType>
      <idpUserId>104</idpUserId>
      <givenName>Marco</givenName>
      <surname>Frei</surname>
      <userEmail>marco.frei1@bug.ch</userEmail>
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.10: addUser - E-Mail nicht gefunden

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Processing add user failed for idpUserId=104</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 5

Beschreibung:

addUser - Es wurden keine Nutzerdaten übermittelt (leere Felder).

Input:

Listing 11.11: addUser-Anfrage - Leere Felder

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType>
      <idpUserId></idpUserId>
      <givenName></givenName>
      <surname></surname>
      <userEmail></userEmail>
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.12: addUser - Verarbeitung fehlgeschlagen

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Processing add user failed for idpUserId=</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

11.4.3 Endpoint: updateUser

Test-ID: 6

Beschreibung:

updateUser - Nutzer wird erfolgreich aktualisiert.

Input:

Listing 11.13: updateUser-Anfrage - Erfolgreiche Aktualisierung

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:updateUserType>
      <idpUserId>200</idpUserId>
      <givenName>Laura</givenName>
      <surname>Schmidt</surname>
      <userEmail>laura.schmidt@pax.ch</userEmail>
    </user:updateUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.14: updateUser - Erfolgreiche Antwort

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 7

Beschreibung:

updateUser - Gesuchter Nutzer nicht vorhanden.

Input:

Listing 11.15: updateUser-Anfrage - Nutzer nicht vorhanden

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:updateUserType>
      <idpUserId>201</idpUserId>
      <givenName>Heidi</givenName>
      <surname>Meier</surname>
    </user:updateUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<userEmail>heidi.meier1@bug.ch</userEmail>
</user:updateUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.16: updateUser - Nutzer nicht gefunden

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_NOT_FOUND</returnCode>
      <errorMessage>User cannot update for idpUserId=201</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 8

Beschreibung:

updateUser - Payload unvollständig (z.B. fehlt Nachname).

Input:

Listing 11.17: updateUser-Anfrage - Unvollständiger Payload

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
  userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:updateUserType>
      <idpUserId>111</idpUserId>
      <givenName>Marco</givenName>
      <userEmail>marco.frei@pax.ch</userEmail>
      <!-- surname fehlt -->
    </user:updateUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.18: updateUser - Fehlerhafte Nutzerdaten

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Payload validation failed for idpUserId=111</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
```

```
</SOAP-ENV:Envelope>
```

Test-ID: 9

Beschreibung:

updateUser - Es wurden keine Nutzerdaten übermittelt (leere Felder).

Input:

Listing 11.19: updateUser-Anfrage - Leere Felder

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:updateUserType>
      <idpUserId></idpUserId>
      <givenName></givenName>
      <surname></surname>
      <userEmail></userEmail>
    </user:updateUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.20: updateUser - Verarbeitung fehlgeschlagen

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnTypes xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_NOT_FOUND</returnCode>
      <errorMessage>User cannot update for idpUserId</errorMessage>
    </ns3:commonReturnTypes>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

11.4.4 Endpoint: deleteUser

Test-ID: 10

Beschreibung:

deleteUser - IDP-Nummer wird erfolgreich gelöscht.

Input:

Listing 11.21: deleteUser-Anfrage - Erfolgreiche Löschung

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:deleteUserType>
      <idpUserId>005</idpUserId>
      <deletionDate>2024-12-01T00:00:00Z</deletionDate>
      <deletionUser>admin</deletionUser>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:10Z</requestTime>
    </user:deleteUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.22: deleteUser - Erfolgreiche Antwort

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonResponseType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
    </ns3:commonResponseType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 11

Beschreibung:

deleteUser - Ungültige IDP-Nummer (z.B. zu lang, nicht vorhanden).

Input:

Listing 11.23: deleteUser-Anfrage - Ungültige IDP-Nummer

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:deleteUserType>
      <idpUserId>11111111111111111111111111111111</idpUserId>
      <deletionDate>2024-12-01T00:00:00Z</deletionDate>
    </user:deleteUserType>
  </soapenv:Body>
</soapenv:Envelope>
```


Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<deletionUser>admin</deletionUser>
<requestId>1</requestId>
<requestTime>2024-12-01T00:00:10Z</requestTime>
</user:deleteUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.24: deleteUser - Fehlerhafte IDP-Nummer

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnTypes xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Payload validation for delete user failed for idpUserId=11111111111111111111</errorMessage>
    </ns3:commonReturnTypes>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 12

Beschreibung:

deleteUser - Nutzer nicht gefunden.

Input:

Listing 11.25: deleteUser-Anfrage - Nutzer nicht gefunden

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
  userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:deleteUserType>
      <idpUserId>999</idpUserId>
      <deletionDate>2024-12-01T00:00:00Z</deletionDate>
      <deletionUser>admin</deletionUser>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:10Z</requestTime>
    </user:deleteUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.26: deleteUser - Nutzer nicht vorhanden

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnTypes xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_NOT_FOUND</returnCode>
    </ns3:commonReturnTypes>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<errorMessage>User to be deleted not found idpUserId=999</errorMessage>
</ns3:commonReturnType>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 13

Beschreibung:

deleteUser - Keine IDP-Nummer übermittelt (leeres Feld).

Input:

Listing 11.27: deleteUser-Anfrage - Leerer Wert

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
  userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:deleteUserType>
      <idpUserId/>
      <deletionDate>2024-12-01T00:00:00Z</deletionDate>
      <deletionUser>admin</deletionUser>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:10Z</requestTime>
    </user:deleteUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.28: deleteUser - Kein Nutzer angegeben

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_NOT_FOUND</returnCode>
      <errorMessage>User to be deleted not found idpUserId=</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

11.4.5 Endpoint: getUserStatus

Test-ID: 14

Beschreibung:

getUserStatus - Nutzer wird erfolgreich gefunden, IDP-Nummer existiert.

Input:

Listing 11.29: getUserStatus-Anfrage - Nutzer vorhanden

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:getUserStatusType>
      <idpUserId>004</idpUserId>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:00Z</requestTime>
    </user:getUserStatusType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.30: getUserStatus - Erfolgreiche Antwort

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:userStatusReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
      <userStatus>OK</userStatus>
    </ns3:userStatusReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 15

Beschreibung:

getUserStatus - Nutzer nicht gefunden.

Input:

Listing 11.31: getUserStatus-Anfrage - Nutzer nicht vorhanden

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:getUserStatusType>
      <idpUserId>999</idpUserId>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:00Z</requestTime>
    </user:getUserStatusType>
  </soapenv:Body>
</soapenv:Envelope>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
</user:getUserStatusType>
</soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.32: getUserStatus - Nutzer nicht gefunden

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:userStatusResponseType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_NOT_FOUND</returnCode>
      <errorMessage>User status not found for idpUserId=999</errorMessage>
    </ns3:userStatusResponseType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 16

Beschreibung:

getUserStatus - Keine IDP-Nummer übergeben (leeres Feld).

Input:

Listing 11.33: getUserStatus-Anfrage - Leere IDP-Nummer

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
  userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:getUserStatusType>
      <idpUserId></idpUserId>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:00Z</requestTime>
    </user:getUserStatusType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.34: getUserStatus - Keine IDP-Nummer übergeben

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:userStatusResponseType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_NOT_FOUND</returnCode>
      <errorMessage>User status not found for idpUserId=</errorMessage>
    </ns3:userStatusResponseType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

11.4.6 Endpoint: addUser, updateUser, deleteUser, getUser

Sollte der Service nicht ansprechbar sein, wird jegliche Anfrage mit einem Fehler beantwortet.

Test-ID: 14

Beschreibung:

addUser, updateUser, deleteUser, getUser Service ist nicht erreichbar.

Input:

Listing 11.35: getUser-Anfrage mit fehlerhaften Daten Input

EGAL

Erwartet:

Listing 11.36: getUser-Anfrage mit fehlerhaften Daten Output

null/empty

11.4.7 Erweiterung der CI/CD-Pipeline mit Docker und Playwright

Zur Sicherstellung der korrekten Zusammenarbeit zwischen der Applikation und dem LDAP-System ist geplant, einen zusätzlichen CI/CD-Job für automatisierte End-to-End-Tests (E2E) zu implementieren. Dieser basiert auf dem bestehenden `docker-compose.yml` und nutzt Playwright zur Ausführung der Tests.

Hinweis: Es handelt sich hierbei um eine konzeptionelle Designüberlegung. Die finale technische Umsetzung kann im Verlauf der Implementierungsphase angepasst oder erweitert werden.

Geplante Testumgebung

Mittels Docker Compose soll eine isolierte Testumgebung aufgesetzt werden, die folgende Komponenten umfasst:

- Die Spring-Boot-Applikation (`pax-ecohub-provisioning`)
- Einen OpenLDAP-Container mit vordefiniertem Schema und Testdaten

Ablauf des neuen Jobs `api-tests`

1. Start der Testumgebung mit `docker-compose up -d`
2. Warten auf Initialisierung der Dienste
3. Ausführen der Playwright-Tests gegen die laufende Umgebung
4. Aufräumen der Umgebung mittels `docker-compose down`

Geplanter CI/CD-Ausschnitt

Listing 11.37: Entwurf: CI/CD-Job für E2E-Tests

```
jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      -uses: actions/checkout@v4

      -name: Set up Docker Compose environment
        run: docker-compose up -d

      -name: Wait for services to initialize
        run: sleep 20

      -name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      -name: Install dependencies and run Playwright tests
        run: |
          npm ci
          npx playwright test

      -name: Tear down environment
        if: always()
        run: docker-compose down
```

Ziele dieser Erweiterung

- Vollständige Integrationstests der Anwendung in Kombination mit LDAP
- Sicherstellung der API-Funktionalität unter realen Bedingungen
- Wiederverwendung der vorhandenen Container-Infrastruktur aus der Entwicklung

11.5 Technische Arbeitspakete

In folgendem Abschnitt werden die technischen Arbeitspakete aufgelistet, die im Rahmen dieses Projekts umgesetzt werden. Es handelt sich hierbei um veränderungen an dem bestehenden Code, keine dokumentationen oder organisationellen Änderungen.

Da die Mapping-Funktion Änderungen an der bestehenden Struktur erfordert, wird diese zuerst umgesetzt.

Tabelle 11.1: Technische Arbeitspakete

ID	Titel	Beschreibung	Dauer
1	Konfigurationsfile erstellen	Es wird ein Konfigurationsfile erstellt, welches die Mapping-Funktion definiert.	1h
2	Entferne Nutzer aus bekannten Gruppen	Es wird eine Funktion implementiert, welche die Nutzer aus den bekannten Gruppen entfernt.	1h
3	Nutzer den neuen Gruppen zuordnen	Es wird eine Funktion implementiert, welche die Nutzer den neuen Gruppen zuordnet.	2h
4	Testen der updateUser-Funktion	Die Funktion wird getestet, ob sie die Mapping-Funktion korrekt umsetzt.	2h
5	API-Tests für Endpunkt addUser erstellen	Es wird eine Playwright-Testsuite erstellt, welche die API-Funktionalität des Endpunktes addUser testet.	1h
6	API-Tests für Endpunkt updateUser erstellen	Es wird eine Playwright-Testsuite erstellt, welche die API-Funktionalität des Endpunktes updateUser testet.	1h
7	API-Tests für Endpunkt deleteUser erstellen	Es wird eine Playwright-Testsuite erstellt, welche die API-Funktionalität des Endpunktes deleteUser testet.	1h
8	API-Tests für Endpunkt getUserStatus erstellen	Es wird eine Playwright-Testsuite erstellt, welche die API-Funktionalität des Endpunktes getUserStatus testet.	1h
9	Integration der Tests in die CI/CD-Pipeline	Es wird die Testsuite in die CI/CD-Pipeline integriert und die Tests werden ausgeführt.	2h

11.6 Tools und Technologien

In diesem Abschnitt werden die im Projekt eingesetzten Tools und Technologien beschrieben. Sie dienen der Unterstützung bei Entwicklung, Test und Qualitätssicherung der erweiterten SOAP-Schnittstellen sowie der LDAP-Anbindung.

11.6.1 Testen der Mapping-Funktion mit SoapUI und OpenLDAP

Die im Rahmen der updateUser-Funktion implementierte rollenbasierte Mapping-Funktion kann nicht vollständig durch automatisierte API-Tests abgedeckt werden. Um diese dennoch zuverlässig zu testen, wird ein manueller Testansatz verfolgt.

Hierzu wird die Anwendung lokal gestartet und mit Hilfe von **SoapUI** gezielt über den updateUser-SOAP-Endpunkt angesprochen. Gleichzeitig wird ein **OpenLDAP**-Container lokal ausgeführt, welcher als Testver-

zeichnisdienst dient. So kann überprüft werden, ob Benutzer - basierend auf übermittelten Rollen - korrekt den entsprechenden Gruppen im LDAP hinzugefügt werden.

SoapUI ermöglicht das flexible Erstellen und Anpassen von SOAP-Anfragen, wodurch verschiedene Rollenkonstellationen getestet werden können. Durch anschließende Kontrolle der Einträge im LDAP (z. B. mit `ldapsearch`) lässt sich die Funktionsweise der Mapping-Logik gezielt überprüfen.

11.6.2 Automatisierte API-Tests mit Playwright

Zur Sicherstellung der allgemeinen Funktionsfähigkeit der implementierten SOAP-Endpunkte wird das Framework **Playwright** eingesetzt. Obwohl Playwright ursprünglich für browserbasierte End-to-End-Tests konzipiert wurde, lässt es sich auch hervorragend für HTTP-basierte API-Tests verwenden.

In diesem Projekt wird Playwright verwendet, um die vier zentralen Funktionen der SOAP-Schnittstelle automatisiert zu testen: `addUser`, `updateUser`, `deleteUser` und `getUserStatus`. Ziel ist es, die korrekte Verarbeitung valider Anfragen zu prüfen sowie typische Fehlerfälle (z. B. fehlende Felder oder ungültige Inhalte) automatisiert zu erkennen. Die Tests werden in TypeScript implementiert und als Teil der CI/CD-Pipeline über GitHub Actions ausgeführt.

Erfahrung im Unternehmen: Bei der **PAX** wird Playwright bereits erfolgreich in anderen Entwicklungsprojekten eingesetzt, insbesondere zur automatisierten Prüfung von Webanwendungen. Die vorhandene Erfahrung im Team sowie bestehende Best Practices sprachen für den Einsatz dieses Tools auch im Rahmen dieses Projekts.

Vergleich mit Alternativen: Vor der Entscheidung für Playwright wurden auch andere Tools in Betracht gezogen, wie etwa:

- **Postman/Newman:** Gute Unterstützung für einfache REST-Tests, aber eingeschränkte SOAP-Funktionalität und weniger flexible Automatisierung.
- **REST Assured (Java):** Sehr leistungsfähig für HTTP-Tests, allerdings eng an die Java-Welt gebunden und weniger etabliert im Team.
- **SoapUI:** Gut geeignet für manuelle Tests, aber weniger flexibel für API-Testautomatisierung in CI/CD.

Die Entscheidung fiel letztlich zugunsten von Playwright, da es bereits im Unternehmen etabliert ist, moderne Entwicklungs-Workflows unterstützt und sich gut in bestehende Pipelines integrieren lässt.

11.6.3 Versionierung

Eine sichere Versionsverwaltung ist besonders wichtig. Sollte es zu Datenverlust oder Fehlern kommen, kann eine saubere Versionierung die Wiederherstellung der letzten funktionierenden Version ermöglichen.

In dieser Arbeit wird für die Versionierung **Git** in Kombination mit der Online-Plattform **GitHub** verwendet. Dies entspricht auch dem Vorgehen im Unternehmen Pax, welches GitHub als Standard-VCS nutzt.

Git ist ein verteiltes Versionskontrollsystem, mit dem Änderungen an Dateien – insbesondere Quellcode – nachvollzogen, rückgängig gemacht oder zusammengeführt werden können. Es erlaubt parallele Entwicklungszweige (Branches) und bietet vollständige Nachvollziehbarkeit aller Änderungen über sogenannte Commits.

GitHub bietet als Cloud-Dienst für Git-Repositories eine intuitive Weboberfläche, Pull Requests, Issue-Tracking, eine Commit-Historie sowie die Integration von CI/CD-Prozessen. Dadurch wird eine strukturierte und nachvollziehbare Entwicklung sichergestellt.

In dieser Arbeit wurde für jedes neue Feature oder jede relevante Änderung ein separater Commit durchgeführt. Dadurch bleibt der Verlauf der Entwicklung klar nachvollziehbar (siehe Abbildung 11.3).

Zudem wurde ein **eigener Branch** mit dem Namen `ipa/mapping-funktion` erstellt, in welchem alle Entwicklungen rund um die Mapping-Funktion implementiert wurden. Nach Abschluss dieser Arbeiten wurde ein **Pull Request** erstellt, um den Branch in den `main`-Zweig zu integrieren. Dies stellt sicher, dass Änderungen nachvollziehbar überprüft und gezielt zusammengeführt werden.

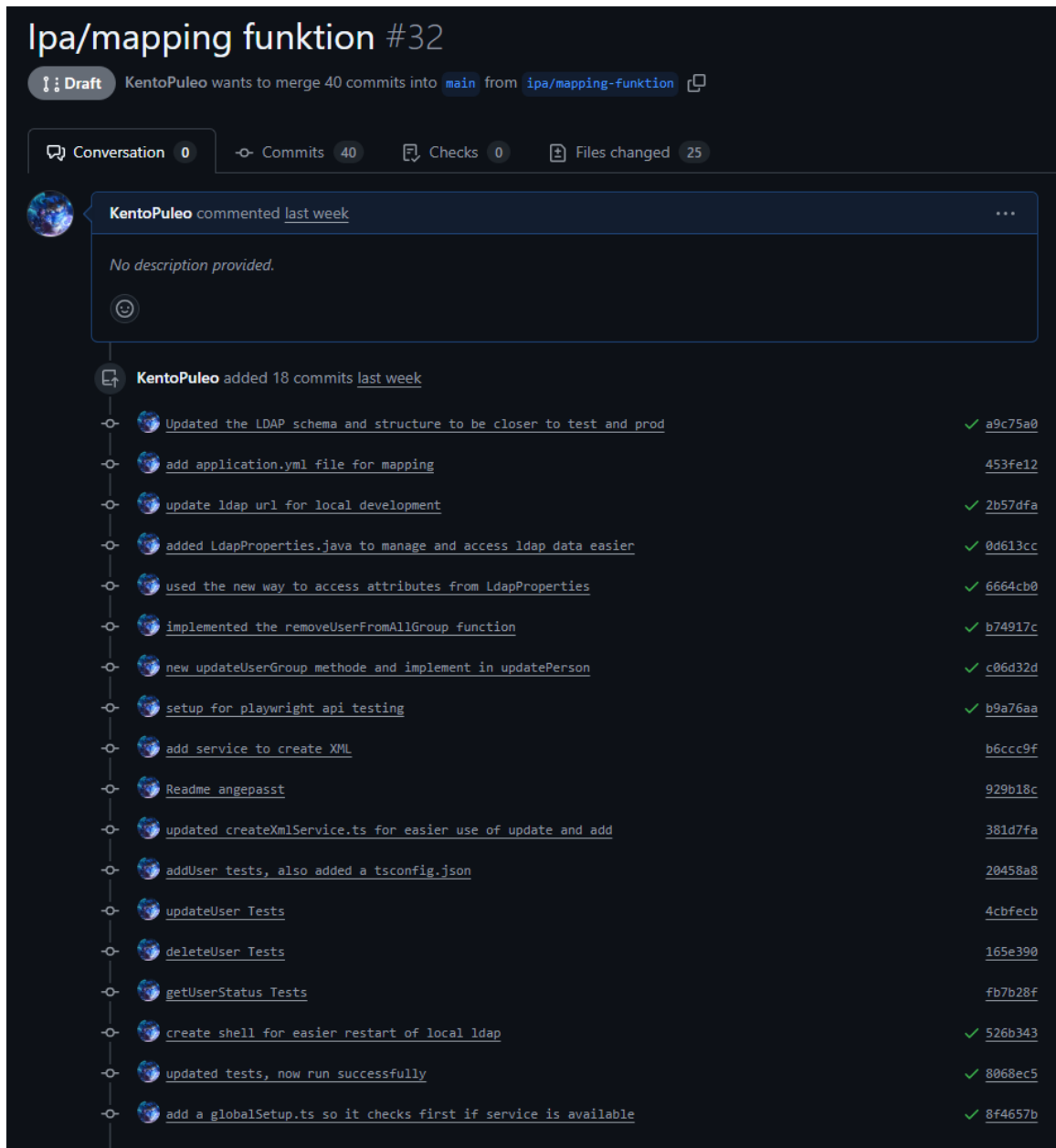


Abbildung 11.3: Pull Request mit strukturierter Commit-Historie im Branch ipa/mapping-funktion

11.7 Clean Code Prinzipien und Namenskonzept

Im Rahmen der Implementierung wurde besonderer Wert auf die Einhaltung von **Clean Code Prinzipien** gelegt. Ziel war es, die Lesbarkeit, Verständlichkeit und Wartbarkeit des Codes sicherzustellen. Die Umsetzung orientierte sich an bewährten Best Practices nach Robert C. Martin („Uncle Bob“) sowie an internen Entwicklungsrichtlinien der PAX.

Die wichtigsten Massnahmen zur Verbesserung der Codequalität umfassten:

- **Sprechende Bezeichner:** Methoden, Variablen und Klassen wurden mit aussagekräftigen Namen versehen, um deren Funktion klar zu kommunizieren (z. B. `mapRolesToGroups()` statt `handle()`).
- **Namenskonzept und Konventionen:** Es wurde ein einheitliches Namenskonzept verwendet:
 - camelCase für Variablen- und Methodennamen (z. B. `userId`, `updateUserEntry()`),
 - PascalCase für Klassennamen (z. B. `PersonRepositoryImpl`),
 - UPPER_SNAKE_CASE für Konstanten (z. B. `LDAP_BASE_DN`).

Diese Konventionen fördern die visuelle Trennung und helfen dabei, unterschiedliche Code-Elemente auf einen Blick zu erkennen.

- **Single Responsibility Principle:** Methoden und Klassen wurden so gestaltet, dass sie jeweils nur eine klar definierte Aufgabe übernehmen. Komplexe Logik wurde in kleinere, gut testbare Einheiten aufgeteilt.
- **Vermeidung von magischen Werten:** Feste Werte wie Gruppen-DNs oder Rollenpräfixe wurden zentral in der `application.yml` oder in Konstanten abgelegt.
- **Konsistente Formatierung:** Der Quellcode wurde gemäss dem PAX-internen Formatierungsstil (basierend auf dem Google Java Style Guide) strukturiert. Dies wurde durch entsprechende IDE-Tools automatisiert unterstützt.
- **Zielgerichtete Fehlerbehandlung:** Fehlerfälle wurden gezielt behandelt und mit verständlichen Logmeldungen versehen, um eine einfache Diagnose und Debugging zu ermöglichen.

Die konsequente Anwendung dieser Prinzipien verbessert nicht nur die Wartbarkeit, sondern erleichtert auch den Wissenstransfer innerhalb des Teams. Neue Entwicklerinnen und Entwickler können sich so schneller im Code zurechtfinden.

KAPITEL 12

IMPLEMENTIERUNG

Das folgende Kapitel wurde in zwei grundlegende Abschnitte unterteilt.

Im ersten Abschnitt wird die Mapping-Funktion behandelt, es wird veranschaulicht, welche neuen Methoden hinzugefügt wurden und wie die bisherigen Methoden verändert wurden. Alle neuen Dateien werden erklärt und mit einem kurzen Code-Schnipsel veranschaulicht.

Im zweiten Abschnitt wird die Testautomation behandelt. Es wird erklärt, wie die einzelnen Endpunkte getestet werden, wie die Tests aufgebaut sind und wie diese automatisiert werden.

12.1 Mapping-Funktion

Die rollenbasierte Gruppenzuweisung ist ein zentraler Bestandteil der Benutzerverwaltung im Rahmen dieser Erweiterung. Sie sorgt dafür, dass Benutzer auf Basis ihrer übermittelten Rollen automatisch den richtigen Gruppen im LDAP-Verzeichnis zugewiesen werden. Die dafür notwendige Konfiguration sowie die Implementierung der entsprechenden Funktionen werden im Folgenden beschrieben.

12.1.1 Konfigurationsdatei

Die Konfigurationsdatei für die Mapping-Funktion befindet sich im Projekt unter dem Namen `application.yml`. In dieser Datei werden die Rollen Kürzel (z. B. `EL_`, `KL_`) den zugehörigen LDAP-Gruppen zugewiesen.

Listing 12.1: Auszug aus der YAML-Konfiguration für rollenbasiertes Mapping

```
ecohub:
  ldap:
    groupSearchBase: ou=groups,dc=paxportal,dc=ch
    roleMappings:
      EL_:
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
-cn=EXTERNPAX_A_BROKERPORTAL,ou=groups
-cn=EXTERNPAX_A_QUICKSALE,ou=groups
-cn=EXTERNPAX_A_LOGISMATA_B2B,ou=groups
-cn=EXTERNPAX_A_BROKERDASHBOARD,ou=groups
KL_:
-cn=EXTERNPAX_A_VERTRAGSSERVICE,ou=groups
```

Die Datei liegt im Ressourcenverzeichnis des Projekts, wie in Abbildung 12.1 dargestellt.

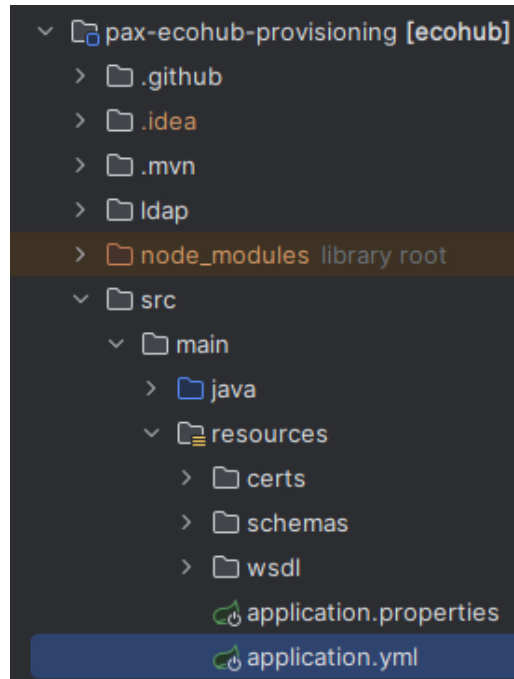


Abbildung 12.1: Pfad zur application.yml-Datei (Quelle: Eigenerarbeitung)

12.1.2 Entfernen des Nutzers aus bestehenden Gruppen

Bevor ein Nutzer auf Basis seiner neuen Rollen den zugehörigen Gruppen zugeordnet werden kann, muss er aus allen Gruppen entfernt werden, in denen er möglicherweise noch eingetragen ist. Dies stellt sicher, dass keine veralteten oder mehrfachen Gruppenzugehörigkeiten bestehen.

Zweck der Methode

Die Methode `removeUserFromAllGroup` entfernt einen Nutzer anhand seines sogenannten *Distinguished Name (DN)* aus allen Gruppen, die über das Rollenkonfigurations-Mapping bekannt sind. Die Methode befindet sich in der Datei `PersonRepositoryImpl.java`.

Aufbau der Methode

Die Methode ist in mehrere sinnvolle Teilschritte gegliedert, die im Folgenden einzeln erklärt werden.

1. Zusammensetzen des vollständigen Benutzer-DN Damit der Nutzer in den LDAP-Gruppen eindeutig identifiziert werden kann, muss sein DN vollständig angegeben werden. Dazu wird an den mitgegebenen DN-Teil der Domänenteil `dc=paxportal,dc=person` angehängt:

Listing 12.2: Konstruktion des vollständigen Benutzer-DN

```
userDn = userDn + "dc=paxportal,dc=person";
```

2. Ermittlung aller bekannten Gruppen Statt alle Gruppen manuell aufzulisten, wird hier dynamisch aus der bestehenden Rollenkonfiguration ('roleMappings') ermittelt, welche Gruppen existieren. So ist die Methode flexibel erweiterbar, ohne Änderungen im Code:

Listing 12.3: Dynamisches Sammeln aller Zielgruppen

```
Set<String> allGroups = new HashSet<>();  
for (List<String> groups : getRoleMappings().values()) {  
    allGroups.addAll(groups);  
}
```

3. Entfernen des Nutzers aus jeder Gruppe Für jede gefundene Gruppe wird versucht, den Nutzer aus dem Gruppenobjekt zu entfernen. Dies geschieht über eine LDAP-Änderungsoperation, bei der der 'member'-Eintrag des Nutzers gelöscht wird. Falls der Nutzer nicht in der Gruppe enthalten ist oder ein Fehler auftritt, wird dies protokolliert, aber die Ausführung geht weiter:

Listing 12.4: LDAP-Entfernungsvorgang für Gruppenmitgliedschaft

```
for (String groupDn : allGroups) {  
    try {  
        ModificationItem[] mods = new ModificationItem[] {  
            new ModificationItem(DirContext.REMOVE_ATTRIBUTE, new BasicAttribute("member", userDn))  
        };  
  
        ldapTemplate.modifyAttributes(groupDn, mods);  
        log.info("Remove user from group [userDn={}, groupDn={}]", userDn, groupDn);  
    } catch (Exception e) {  
        log.debug("User not in group or other error. [userDn={}, groupDn={}, exception={}]",  
            userDn, groupDn, e.getMessage());  
    }  
}
```

4. Rückmeldung über Erfolg oder Fehler Zum Schluss gibt die Methode einen `boolean`-Wert zurück. Dieser signalisiert, ob der gesamte Entfernungsvorgang erfolgreich war oder ob ein Fehler auftrat:

Listing 12.5: Fehlerbehandlung und Rückgabewert

```
log.info("Remove user from group [userDn={}]", userDn);  
return true;  
} catch (Exception e) {  
    log.error("Failed to remove user from groups. [userDn={}, exception={}]", userDn, e.getMessage());  
    return false;  
}
```

Fazit

Die Methode `removeUserFromAllGroup` sorgt für einen sauberen Ausgangszustand, indem sie alle bestehenden Gruppenzugehörigkeiten eines Nutzers entfernt. Damit wird sichergestellt, dass nachfolgende Zuweisungen (siehe nächste Subsection) konsistent und fehlerfrei erfolgen können. Durch die dynamische Ermittlung der Gruppen ist die Methode flexibel und wartungsarm.

Die komplette removeUserFromAllGroup Methode

Hier die komplette removeUserFromAllGroup Methode:

Listing 12.6: Die komplette removeUserFromAllGroup Methode

```
private boolean removeUserFromAllGroup(String userDn) {
    userDn = userDn + "dc=paxportal,dc=person";
    try {
        Set<String> allGroups = new HashSet<>();
        for (List<String> groups : getRoleMappings().values()) {
            allGroups.addAll(groups);
        }

        for (String groupDn : allGroups) {
            try {
                ModificationItem[] mods = new ModificationItem[] {
                    new ModificationItem(DirContext.REMOVE_ATTRIBUTE, new BasicAttribute("member", userDn))
                };

                ldapTemplate.modifyAttributes(groupDn, mods);
                log.info("Remove user from group [userDn={}, groupDn={}]", userDn, groupDn);
            } catch (Exception e) {
                log.debug("User not in group or other error. [userDn={}, groupDn={}, exception={}]",
                    userDn, groupDn, e.getMessage());
            }
        }

        log.info("Remove user from group [userDn={}]", userDn);
        return true;
    } catch (Exception e) {
        log.error("Failed to remove user from groups. [userDn={}, exception={}]", userDn, e.getMessage());
        return false;
    }
}
```

12.1.3 Hinzufügen des Nutzers zu den neuen Gruppen

Nachdem der Nutzer aus allen bestehenden LDAP-Gruppen entfernt wurde (siehe Abschnitt 12.1.2), wird er in einem zweiten Schritt wieder den richtigen Gruppen zugewiesen. Dies geschieht auf Basis der Rollen, die im sogenannten CompleteUserType-Objekt übermittelt werden. Die zentrale Methode für diese Aufgabe heisst updateUserGroup und befindet sich in der Datei PersonRepositoryImpl.java.

Zweck der Methode

Die Methode analysiert die Rollen des Nutzers (igRoles), vergleicht sie mit der Konfiguration aus der application.yml, und fügt den Nutzer dann den passenden Gruppen im LDAP hinzu.

Aufbau der Methode

Im Folgenden werden die wichtigsten Bestandteile der Methode schrittweise erklärt und jeweils mit einem Codeausschnitt ergänzt.

1. Vorbereitung und Abbruch bei fehlenden Rollen Zu Beginn wird überprüft, ob der Nutzer überhaupt Rollen besitzt. Falls keine vorhanden sind, wird der Vorgang übersprungen:

Listing 12.7: Abbruch bei fehlenden Rolleninformationen

```
if (payload == null || payload.getIgRoles() == null || payload.getIgRoles().isEmpty()) {  
    log.info("No igRoles provided, skipping group updates");  
    return;  
}
```

2. Ermittlung der Zielgruppen basierend auf Rollenpräfixen Es wird eine Liste von Zielgruppen aufgebaut. Dafür werden alle Rollen mit den definierten Präfixen (z. B. EL_, KL_) verglichen. Jede passende Rolle führt zur Zuordnung der zugehörigen Gruppen:

Listing 12.8: Zuordnung der Rollen zu Gruppen

```
Map<String, Boolean> targetGroups = new HashMap<>();  
List<String> igRoles = payload.getIgRoles();  
  
for (String igRole : igRoles) {  
    for (Map.Entry<String, List<String>> entry : getRoleMappings().entrySet()) {  
        String prefix = entry.getKey();  
        List<String> groups = entry.getValue();  
  
        if (igRole.startsWith(prefix)) {  
            for (String groupDn : groups) {  
                targetGroups.put(groupDn, true);  
            }  
        }  
    }  
}
```

3. Nutzer wird zu jeder relevanten Gruppe hinzugefügt Für jede ermittelte Gruppe wird eine LDAP-Operation ausgeführt, die den Nutzer der Gruppe hinzufügt. Diese Operation wird mit einer sogenannten ModificationItem-Anweisung durchgeführt:

Listing 12.9: Hinzufügen des Nutzers zu den LDAP-Gruppen

```
for (String groupDn : targetGroups.keySet()) {  
    try {  
        ModificationItem[] mods = new ModificationItem[] {  
            new ModificationItem(DirContext.ADD_ATTRIBUTE, new BasicAttribute("member", userDn))  
        };  
    }
```

```
        ldapTemplate.modifyAttributes(groupDn, mods);
        log.info("Added user to group. [userDn={}, groupDn={}]", userDn, groupDn);
    } catch (Exception e) {
        log.error("Failed to add user to group. [userDn={}, groupDn={}, exception={}, cause={}]",
            userDn, groupDn, e.getMessage(), e.getCause() != null ? e.getCause().getMessage() : "none");
    }
}
```

4. Fehlerbehandlung Die gesamte Methode ist zusätzlich in einen try-catch-Block gehüllt, um bei allgemeinen Fehlern während der Gruppenverarbeitung ein Logging vorzunehmen:

Listing 12.10: Allgemeine Fehlerbehandlung am Ende der Methode

```
} catch (Exception e) {
    log.error("Failed to update user groups. [userDn={}, exception={}]", userDn, e.getMessage());
}
```

Fazit

Die Methode `updateUserGroup` ist zentral für die automatische rollenbasierte Gruppensteuerung im LDAP. Sie stellt sicher, dass ein Nutzer nach dem Update nur noch in den Gruppen enthalten ist, die seiner aktuellen Rollenstruktur entsprechen. Dabei wird auf Konfigurierbarkeit über die YAML-Datei gesetzt, wodurch die Lösung flexibel erweitert werden kann.

Die komplette updateUserGroup Methode

Hier die komplette updateUserGroup Methode:

Listing 12.11: Die komplette updateUserGroup Methode

```
private void updateUserGroup(String userDn, final CompleteUserType payload) {
    userDn = userDn + ",dc=paxportal,dc=ch";
    if (payload == null || payload.getIgRoles() == null || payload.getIgRoles().isEmpty()) {
        log.info("No igRoles provided, skipping group updates");
        return;
    }

    try {
        Map<String, Boolean> targetGroups = new HashMap<>();
        List<String> igRoles = payload.getIgRoles();

        for (String igRole : igRoles) {
            for (Map.Entry<String, List<String>> entry : getRoleMappings().entrySet()) {
                String prefix = entry.getKey();
                List<String> groups = entry.getValue();

                if (igRole.startsWith(prefix)) {
                    for (String groupDn : groups) {
                        targetGroups.put(groupDn, true);
                    }
                }
            }
        }

        for (String groupDn : targetGroups.keySet()) {
            try {
                ModificationItem[] mods = new ModificationItem[] {
                    new ModificationItem(DirContext.ADD_ATTRIBUTE, new BasicAttribute("member", userDn))
                };

                log.debug("Attempting to add user to group. [userDn={}, groupDn={}]", userDn, groupDn);
                ldapTemplate.modifyAttributes(groupDn, mods);
                log.info("Added user to group. [userDn={}, groupDn={}]", userDn, groupDn);
            } catch (Exception e) {
                log.error("Failed to add user to group. [userDn={}, groupDn={}, exception={}, cause={}]",
                    userDn, groupDn, e.getMessage(), e.getCause() != null ? e.getCause().getMessage() : "none");
            }
        }

        log.info("Remove user from group [userDn={}]", userDn);
    } catch (Exception e) {
        log.error("Failed to update user groups. [userDn={}, exception={}]", userDn, e.getMessage());
    }
}
```

12.1.4 Einbindung der Mapping-Funktionen in updateUser

Die zuvor beschriebenen Methoden `removeUserFromAllGroup` und `updateUserGroup` werden innerhalb der zentralen `updateUser`-Funktion aufgerufen. Diese Funktion wird verwendet, wenn bestehende Benutzer im LDAP-Verzeichnis aktualisiert werden sollen.

Dabei erfolgt die Gruppenverwaltung in drei aufeinanderfolgenden Schritten:

1. Änderung der Basisattribute des Benutzers (z. B. IDP-Nummer)
2. Entfernen des Benutzers aus allen Gruppen
3. Neue Gruppenzuweisung basierend auf den aktuellen Rollen

Die folgende Methode zeigt die vollständige Integration:

Listing 12.12: updateUser-Methode mit eingebauten Mapping-Funktionen

```
@Override
public boolean updatePerson(final Person person, final CompleteUserType payload) {
    final String idpUserId = payload.getIdpUserId();
    boolean updated = false;
    log.info("Update person in LDAP started. [idpUserId={}]", idpUserId);

    try {
        ldapTemplate.modifyAttributes(person.getDn(), buildModification(idpUserId));

        boolean removalSuccess = removeUserFromAllGroup(person.getDn().toString());
        if (!removalSuccess) {
            log.error("Failed to remove user from groups. [idpUserId={}]", idpUserId);
            return false;
        }

        updateUserGroup(person.getDn().toString(), payload);

        log.info("Update person in LDAP succeeded. [idpUserId={}]", idpUserId);
        updated = true;

    } catch (final Exception e) {
        log.info("Update person in LDAP failed. [idpUserId={}, exception={}]", idpUserId, e.getMessage());
    }

    log.info("Update person in LDAP finished. [idpUserId={}]", idpUserId);
    return updated;
}
```

Erläuterung der Abläufe

- `modifyAttributes(...)`: Aktualisiert z. B. die IDP-Nummer des Nutzers im LDAP.
- `removeUserFromAllGroup(...)`: Entfernt den Nutzer aus allen Gruppen, basierend auf dem aktuellen Rollenkontext.

- `updateUserGroup(...)`: Fügt den Nutzer anhand der Rollen (`igRoles`) den in der YAML-Konfiguration definierten Gruppen hinzu.

Diese Einbettung stellt sicher, dass jeder Benutzer nach einem Update nur genau in den Gruppen enthalten ist, die seiner aktuellen Rolle entsprechen.

12.2 Implementierung der Tests

12.2.1 Einrichtung und Konfiguration des Testframeworks

Für die automatisierte Überprüfung der SOAP-Schnittstellen wurde das Framework **Playwright** verwendet. Obwohl Playwright primär für End-to-End-Tests von Webanwendungen konzipiert ist, eignet es sich auch hervorragend für API-Tests. Besonders hervorzuheben sind die einfache Handhabung, moderne Syntax sowie eine integrierte Testumgebung.

Im ersten Schritt wurde Playwright lokal ins Projekt integriert:

Listing 12.13: Installation von Playwright

```
npm install -D @playwright/test
npx playwright install
```

Nach der Installation wurde eine geeignete Projektstruktur für die Testautomation aufgebaut:

- `api-tests/` - Hauptverzeichnis für die Playwright-Testfälle.
- `addUser.spec.ts` - Test für den `addUser`-Endpunkt.
- `updateUser.spec.ts` - Test für den `updateUser`-Endpunkt.
- `deleteUser.spec.ts` - Test für den `deleteUser`-Endpunkt.
- `getUserStatus.spec.ts` - Test für den `getUserStatus`-Endpunkt.

Diese Struktur ermöglicht es, die Testfälle übersichtlich zu organisieren und flexibel auf unterschiedliche Testanforderungen einzugehen.

12.2.2 Generierung der SOAP-Requests

Zur Erzeugung valider und ungültiger SOAP-Anfragen wurde im Verzeichnis `api-tests/services` ein separater Service namens `createXmlService.ts` implementiert. Dieser stellt Funktionen bereit, die dynamisch XML-Anfragen für alle unterstützten Endpunkte erzeugen.

Die generierten XMLs basieren auf dem offiziellen Schema der Schnittstelle (`userProvisioningSchemaTypes-v3.0`) und können mit konkreten Nutzerdaten befüllt werden. Dadurch wird sichergestellt, dass die Tests sowohl syntaktisch korrekte als auch gezielt fehlerhafte Payloads erzeugen können.

Der Service umfasst folgende Funktionen:

- `createAddUserXML({...})` - erzeugt eine gültige `addUser`-Anfrage
- `createUpdateUserXML({...})` - erzeugt eine gültige `updateUser`-Anfrage
- `createDeleteUserXML({...})` - erzeugt eine gültige `deleteUser`-Anfrage
- `createGetUserStatusXML({...})` - erzeugt eine gültige `getUserStatus`-Anfrage
- sowie entsprechende Varianten mit absichtlich ungültigen Nutzerdaten

Durch diese Trennung von Testlogik und Payload-Erstellung wird der Code wartbar gehalten und Wiederverwendbarkeit gefördert.

Beispiel: Generierung einer `deleteUser`-SOAP-Anfrage

Nachfolgend ist die Funktion `createDeleteUserXML` dargestellt, die ein vollständiges SOAP-Request für den `deleteUser`-Endpunkt erzeugt. Sie wird in den Tests genutzt, um dynamisch Anfragen mit beliebigen Testdaten zu erstellen:

Listing 12.14: Funktion zur Generierung einer `deleteUser`-SOAP-Anfrage

```
export function createDeleteUserXML(data: {  
  idpUserId: string;  
  deletionDate: string;  
  deletionUser: string;  
  requestId: string;  
  requestTime: string;  
}): string {  
  return `  
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/  
  userProvisioningSchemaTypes-v3.0/">  
  <soapenv:Header/>  
  <soapenv:Body>  
    <user:deleteUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">  
      <idpUserId>${data.idpUserId}</idpUserId>  
      <deletionDate>${data.deletionDate}</deletionDate>  
      <deletionUser>${data.deletionUser}</deletionUser>  
      <requestId>${data.requestId}</requestId>  
      <requestTime>${data.requestTime}</requestTime>  
    </user:deleteUserType>  
  </soapenv:Body>  
</soapenv:Envelope>  
'  
  .trim();  
}
```

Diese Funktion kann z. B. wie folgt aufgerufen werden:

Listing 12.15: Beispielhafte Nutzung in einem Test

```
const xml = createDeleteUserXML({  
  idpUserId: '123456',  
  deletionDate: '2025-05-16',  
  deletionUser: 'testuser',  
});
```

```
requestId: 'REQ-001',  
requestTime: '2025-05-16T10:00:00Z'  
});
```

12.2.3 Test: addUser

Die folgenden Playwright-Tests überprüfen den SOAP-Endpunkt `addUser`, mit dem neue Benutzer in das System aufgenommen werden. Es werden dabei verschiedene Situationen getestet: von einem erfolgreichen Hinzufügen bis hin zu Fehlerfällen bei ungültigem oder unvollständigem Input.

Testfall TC1 - Erfolgreiches Hinzufügen eines Nutzers

Listing 12.16: TC1: Nutzer wird erfolgreich hinzugefügt

```
test('TC1: addUser -IDP Nummer erfolgreich vergeben', async ({ request }) => {  
  const xml = createUserXML({  
    ...basePayload,  
    idpUserId: '100',  
    userEmail: 'marco.frei@pax.ch',  
    givenName: 'Marco',  
    surname: 'Frei',  
  });  
  
  const response = await request.post(endpoint, {  
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },  
    data: xml,  
  });  
  
  const body = await response.text();  
  expect(response.ok()).toBeTruthy();  
  expect(body).toContain('<returnCode>OK</returnCode>');  
});
```

Erklärung: Hier wird ein vollständiger und korrekter SOAP-Request erstellt. Die Antwort des Servers enthält den Code `OK`, was bestätigt, dass der Benutzer erfolgreich hinzugefügt wurde.

Testfall TC2 - Benutzer existiert bereits

Listing 12.17: TC2: IDP Nummer ist bereits vergeben

```
test('TC2: addUser -IDP Nummer bereits vergeben', async ({ request }) => {  
  const xml = createUserXML({  
    ...basePayload,  
    idpUserId: '101',  
    userEmail: 'max.mustermann@pax.ch',  
    givenName: 'Max',  
    surname: 'Mustermann',  
  });  
});
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
const response = await request.post(endpoint, {
  headers: { 'Content-Type': 'text/xml;charset=UTF-8' },
  data: xml,
});

const body = await response.text();
expect(body).toContain('<returnCode>USER_EXISTS</returnCode>');
expect(body).toContain('User already exists with idpUserId=101');
});
```

Erklärung: In diesem Fall wird ein Benutzer mit einer bereits vergebenen IDP-Nummer hinzugefügt. Der Server erkennt dies und gibt eine entsprechende Fehlermeldung zurück.

Testfall TC3 - Unvollständige Nutzerdaten

Listing 12.18: TC3: Nachname fehlt im Payload

```
test('TC3: addUser -unvollständiger Payload', async ({ request }) => {
  const partialPayload = { ...basePayload };
  delete (partialPayload as any).surname;

  const xml = createInvalidUserXML({
    ...partialPayload,
    idpUserId: '103',
    userEmail: 'marco.frei@pax.ch',
    givenName: 'Marco',
  });

  const response = await request.post(endpoint, {
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },
    data: xml,
  });

  const body = await response.text();
  expect(body).toContain('<returnCode>ERROR</returnCode>');
  expect(body).toContain('Payload validation failed for idpUserId=103');
});
```

Erklärung: Hier wird der Nachname bewusst weggelassen. Die Anwendung reagiert mit einem ERROR-Code und gibt einen validierungsspezifischen Fehlertext zurück.

Testfall TC4 - Fehlerhafte E-Mail-Adresse

Listing 12.19: TC4: Technische Prüfung schlägt fehl

```
test('TC4: addUser -nicht existierende Email-Adresse', async ({ request }) => {
  const xml = createUserXML({
    ...basePayload,
    idpUserId: '104',
```


Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
    userEmail: 'marco.freil@bug.ch',
    givenName: 'Marco',
    surname: 'Frei',
  });

  const response = await request.post(endpoint, {
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },
    data: xml,
  });

  const body = await response.text();
  expect(body).toContain('<returnCode>ERROR</returnCode>');
  expect(body).toContain('Processing add user failed for idpUserId=104');
});
```

Erklärung: Obwohl der Payload technisch vollständig ist, wird eine nicht zugelassene oder nicht konfigurierte E-Mail-Adresse verwendet. Die Applikation meldet einen internen Fehler beim Hinzufügen des Nutzers.

Testfall TC5 - Leere Eingabewerte

Listing 12.20: TC5: Leere Pflichtfelder

```
test('TC5: addUser -no value', async ({ request }) => {
  const xml = createUserXML({
    ...basePayload,
    idpUserId: '',
    userEmail: '',
    givenName: '',
    surname: '',
  });

  const response = await request.post(endpoint, {
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },
    data: xml,
  });

  const body = await response.text();
  expect(body).toContain('<returnCode>ERROR</returnCode>');
  expect(body).toContain('Processing add user failed for idpUserId=');
});
```

Erklärung: Dieser Fall testet, wie das System auf komplett leere Felder reagiert. Erwartet wird eine Fehlermeldung, dass der Nutzer nicht verarbeitet werden konnte.

Fazit

Durch die fünf Testfälle wird die gesamte Bandbreite des addUser-Endpunkts abgedeckt. Die Tests prüfen sowohl positive Szenarien als auch typische Fehlerfälle bei der Eingabe. Dadurch ist sichergestellt, dass die Schnittstelle robust und fehlerresistent ist.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Testfallübersicht

Insgesamt wurden fünf Testfälle implementiert, darunter:

- **TC1** - Erfolgreiches Hinzufügen eines Nutzers mit vollständigen und gültigen Daten
- **TC2** - Fehler bei fehlendem Pflichtfeld `surname`
- **TC3** - Fehler bei falschem Format der E-Mail-Adresse
- **TC4** - Fehlerhafte IDP-Nummer (z.B. Buchstaben statt Zahlen)
- **TC5** - Leerer SOAP-Body (technischer Fehlerfall)

Die Tests wurden so gestaltet, dass sie die Robustheit und Fehlertoleranz der Schnittstelle systematisch prüfen.

12.2.4 Test: updateUser

Die folgenden Playwright-Tests überprüfen den SOAP-Endpoint updateUser, mit dem bestehende Benutzer aktualisiert werden. Es werden verschiedene Szenarien abgedeckt - vom erfolgreichen Update bis hin zu typischen Fehlerfällen wie ungültige Eingaben oder unbekannte Nutzer.

Testfall TC6 - Erfolgreiches Aktualisieren eines Nutzers

Listing 12.21: TC6: Nutzer wird erfolgreich aktualisiert

```
test('TC6: updateUser -IDP Nummer erfolgreich aktualisiert', async ({ request }) => {  
  const xml = createUserXML({  
    ...basePayload,  
    idpUserId: '200',  
    userEmail: 'laura.schmidt@pax.ch',  
    givenName: 'Laura',  
    surname: 'Schmidt',  
  });  
  
  const response = await request.post(endpoint, {  
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },  
    data: xml,  
  });  
  
  const body = await response.text();  
  expect(body).toContain('<returnCode>OK</returnCode>');  
});
```

Erklärung: In diesem Test wird ein bereits vorhandener Benutzer erfolgreich aktualisiert. Die Rückmeldung vom Server enthält den Code OK.

Testfall TC7 - Nutzer nicht gefunden

Listing 12.22: TC7: Benutzer mit IDP Nummer existiert nicht

```
test('TC7: updateUser -Nutzer nicht gefunden', async ({ request }) => {  
  const xml = createUserXML({  
    ...basePayload,  
    idpUserId: '201',  
    userEmail: 'heidi.meier1@bug.ch',  
    givenName: 'Heidi',  
    surname: 'Meier',  
  });  
  
  const response = await request.post(endpoint, {  
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },  
    data: xml,  
  });  
  
  const body = await response.text();
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
expect(body).toContain('<returnCode>USER_NOT_FOUND</returnCode>');  
expect(body).toContain('User cannot update for idpUserId=201');  
});
```

Erklärung: Der Test simuliert einen Update-Versuch für eine IDP-Nummer, die im System nicht vorhanden ist. Die Antwort weist den Fehlercode USER_NOT_FOUND aus.

Testfall TC8 - Unvollständige Nutzerdaten

Listing 12.23: TC8: Nachname fehlt im Payload

```
test('TC8: updateUser -unvollständiger Payload', async ({ request }) => {  
  const partialPayload = { ...basePayload };  
  delete (partialPayload as any).surname;  
  
  const xml = createInvalidUserXML({  
    ...partialPayload,  
    idpUserId: '111',  
    userEmail: 'marco.frei@pax.ch',  
    givenName: 'Marco',  
  });  
  
  const response = await request.post(endpoint, {  
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },  
    data: xml,  
  });  
  
  const body = await response.text();  
  expect(body).toContain('<returnCode>ERROR</returnCode>');  
  expect(body).toContain('Payload validation failed for idpUserId=111');  
});
```

Erklärung: Es fehlt ein Pflichtfeld (Nachname), wodurch die Validierung fehlschlägt. Der Server liefert einen Fehlercode ERROR mit entsprechendem Hinweistext.

Testfall TC9 - Leere Eingabewerte

Listing 12.24: TC9: Leere Pflichtfelder im Payload

```
test('TC9: updateUser -no value', async ({ request }) => {  
  const xml = createUserXML({  
    ...basePayload,  
    idpUserId: '',  
    userEmail: '',  
    givenName: '',  
    surname: '',  
  });  
  
  const response = await request.post(endpoint, {  
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },  
  });
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
    data: xml,  
  });  
  
  const body = await response.text();  
  expect(body).toContain('<returnCode>USER_NOT_FOUND</returnCode>');  
  expect(body).toContain('User cannot update for idpUserId=');  
});
```

Erklärung: Der Test überprüft das Verhalten bei komplett leeren Pflichtfeldern. Der Server erkennt, dass kein gültiger Benutzer identifiziert werden kann und reagiert mit dem Fehlercode `USER_NOT_FOUND`.

Fazit

Die Tests für `updateUser` decken zentrale Anwendungsfälle und Fehlerszenarien ab. Sie zeigen, dass sowohl korrekte Updates als auch fehlerhafte Daten eindeutig und nachvollziehbar vom System verarbeitet werden.

Testfallübersicht

Insgesamt wurden vier Testfälle implementiert:

- **TC6** - Erfolgreiches Aktualisieren eines Nutzers
- **TC7** - Nutzer mit gegebener IDP-Nummer existiert nicht
- **TC8** - Pflichtfeld `surname` fehlt im Payload
- **TC9** - Leere Werte führen zu fehlender Identifikation des Nutzers

Die Tests stellen sicher, dass die Update-Logik sowohl funktional als auch validierungsseitig korrekt arbeitet.

12.2.5 Test: deleteUser

Die folgenden Playwright-Tests überprüfen den SOAP-Endpoint deleteUser, mit dem Benutzer aus dem System entfernt werden. Es werden sowohl erfolgreiche Löschungen als auch typische Fehlerszenarien getestet - etwa wenn der Benutzer nicht existiert oder die Eingabe ungültig ist.

Testfall TC10 - Erfolgreiches Löschen eines Nutzers

Listing 12.25: TC10: IDP-Nummer erfolgreich gelöscht

```
test('TC10: deleteUser -IDP-Nummer erfolgreich gelöscht', async ({ request }) => {
  const xml = createDeleteUserXML({
    ...baseData,
    idpUserId: '005',
  });

  const response = await request.post(endpoint, {
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },
    data: xml,
  });

  const body = await response.text();
  expect(response.ok()).toBeTruthy();
  expect(body).toContain('<returnCode>OK</returnCode>');
});
```

Erklärung: Ein Benutzer mit der IDP-Nummer 005 wird erfolgreich gelöscht. Die Serverantwort bestätigt dies mit dem Code OK.

Testfall TC11 - Ungültige oder zu lange IDP-Nummer

Listing 12.26: TC11: Ungültige IDP-Nummer im Payload

```
test('TC11: deleteUser -Payload unvollständig oder ungültig', async ({ request }) => {
  const xml = createDeleteUserXML({
    ...baseData,
    idpUserId: '11111111111111111111111111111111',
  });

  const response = await request.post(endpoint, {
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },
    data: xml,
  });

  const body = await response.text();
  expect(body).toContain('<returnCode>ERROR</returnCode>');
  expect(body).toContain(
    'Payload validation for delete user failed for idpUserId=11111111111111111111111111111111'
  );
});
```

Erklärung: In diesem Test ist die IDP-Nummer ungültig - sie ist zu lang oder entspricht nicht dem erwarteten Format. Der Server gibt einen Fehlercode ERROR zurück.

Testfall TC12 - Benutzer nicht gefunden

Listing 12.27: TC12: Nutzer nicht vorhanden

```
test('TC12: deleteUser -Nutzer nicht gefunden', async ({ request }) => {  
  const xml = createDeleteUserXML({  
    ...baseData,  
    idpUserId: '999',  
  });  
  
  const response = await request.post(endpoint, {  
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },  
    data: xml,  
  });  
  
  const body = await response.text();  
  expect(body).toContain('<returnCode>USER_NOT_FOUND</returnCode>');  
  expect(body).toContain('User to be deleted not found idpUserId=999');  
});
```

Erklärung: Der Benutzer mit der angegebenen IDP-Nummer existiert nicht. Die Anwendung reagiert mit dem spezifischen Fehlercode USER_NOT_FOUND.

Testfall TC13 - Leerer Wert im Payload

Listing 12.28: TC13: Leerer Wert bei IDP-Nummer

```
test('TC13: deleteUser -no value', async ({ request }) => {  
  const xml = createDeleteUserXML({  
    ...baseData,  
    idpUserId: '',  
  });  
  
  const response = await request.post(endpoint, {  
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },  
    data: xml,  
  });  
  
  const body = await response.text();  
  expect(body).toContain('<returnCode>USER_NOT_FOUND</returnCode>');  
  expect(body).toContain('User to be deleted not found idpUserId=');  
});
```

Erklärung: Hier wird der Löschbefehl mit einer leeren IDP-Nummer gesendet. Der Server kann keinen Nutzer identifizieren und gibt den Fehlercode USER_NOT_FOUND zurück.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Fazit

Die Tests stellen sicher, dass der Endpunkt `deleteUser` korrekt funktioniert - sowohl bei gültiger als auch bei fehlerhafter Eingabe. Durch gezielte Fehlersimulationen wird überprüft, ob die API erwartungsgemäß reagiert.

Testfallübersicht

Es wurden vier Testfälle für diesen Endpunkt implementiert:

- **TC10** - Erfolgreiches Löschen eines Nutzers
- **TC11** - Ungültige IDP-Nummer im Payload
- **TC12** - Nutzer mit gegebener IDP-Nummer existiert nicht
- **TC13** - Leerer Wert bei IDP-Nummer

Die Abdeckung umfasst funktionale und validierungsbezogene Anforderungen an die Schnittstelle.

12.2.6 Test: getUserStatus

Die folgenden Playwright-Tests prüfen den SOAP-Endpunkt `getUserStatus`, der verwendet wird, um den aktuellen Status eines Benutzers anhand seiner IDP-Nummer zu ermitteln. Dabei werden sowohl erfolgreiche als auch fehlgeschlagene Abfragen getestet.

Testfall TC14 - Nutzer erfolgreich gefunden

Listing 12.29: TC14: Nutzerstatus erfolgreich abgerufen

```
test('TC14: getUserStatus -Nutzer erfolgreich gefunden', async ({ request }) => {  
  const xml = createGetUserStatusXML({  
    ...baseData,  
    idpUserId: '004',  
  });  
  
  const response = await request.post(endpoint, {  
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },  
    data: xml,  
  });  
  
  const body = await response.text();  
  expect(response.ok()).toBeTruthy();  
  expect(body).toContain('<returnCode>OK</returnCode>');  
  expect(body).toContain('<userStatus>OK</userStatus>');  
});
```

Erklärung: In diesem Test wird der Status eines existierenden Nutzers abgefragt. Der Server gibt den Rückgabecode `OK` sowie den Benutzerstatus `OK` aus.

Testfall TC15 - Nutzer nicht gefunden

Listing 12.30: TC15: Nutzer mit IDP-Nummer existiert nicht

```
test('TC15: getUserStatus -Nutzer nicht gefunden', async ({ request }) => {  
  const xml = createGetUserStatusXML({  
    ...baseData,  
    idpUserId: '999',  
  });  
  
  const response = await request.post(endpoint, {  
    headers: { 'Content-Type': 'text/xml;charset=UTF-8' },  
    data: xml,  
  });  
  
  const body = await response.text();  
  expect(body).toContain('<returnCode>USER_NOT_FOUND</returnCode>');  
  expect(body).toContain('User status not found for idpUserId=999');  
});
```

Erklärung: Dieser Testfall prüft die Reaktion auf eine Anfrage für einen nicht vorhandenen Benutzer. Der Server gibt korrekt den Fehlercode `USER_NOT_FOUND` zurück.

Testfall TC16 - Leerer Wert im Payload

Listing 12.31: TC16: Leerer Wert bei IDP-Nummer

```
test('TC16: getUserStatus -no value', async ({ request }) => {
  const xml = createGetUserStatusXML({
    ...baseData,
    idpUserId: '',
  });

  const response = await request.post(endpoint, {
    headers: { 'Content-Type': 'text/xml; charset=UTF-8' },
    data: xml,
  });

  const body = await response.text();
  expect(body).toContain('<returnCode>USER_NOT_FOUND</returnCode>');
  expect(body).toContain('User status not found for idpUserId=');
});
```

Erklärung: In diesem Test wird die IDP-Nummer leer gelassen. Der Server erkennt, dass keine gültige Benutzer-ID vorhanden ist, und liefert ebenfalls den Fehlercode `USER_NOT_FOUND` zurück.

Fazit

Die Tests für den Endpunkt `getUserStatus` gewährleisten, dass sowohl erfolgreiche Statusabfragen als auch fehlerhafte Anfragen - z. B. bei unbekannter oder leerer IDP-Nummer - zuverlässig verarbeitet werden.

Testfallübersicht

Es wurden drei Testfälle für diesen Endpunkt implementiert:

- **TC14** - Erfolgreiche Statusabfrage für einen bekannten Nutzer
- **TC15** - Anfrage für eine nicht existierende IDP-Nummer
- **TC16** - Leerer Wert bei IDP-Nummer

Somit ist auch für diesen Endpunkt eine saubere Validierung und Fehlertoleranz sichergestellt.

12.2.7 Automatisiertes Zurücksetzen der LDAP-Testumgebung

Da durch die Ausführung der Testfälle Benutzer im LDAP-System angelegt, aktualisiert oder gelöscht werden, musste nach jedem Testlauf ein definierter Ausgangszustand wiederhergestellt werden. Um diesen Prozess zu vereinfachen, wurde ein Bash-Skript entwickelt, welches die Testumgebung automatisiert neu aufsetzt.

Zielsetzung

Das Skript stellt sicher, dass:

- der OpenLDAP-Container neu gestartet wird,
- alle relevanten LDIF-Dateien neu importiert werden (Schema, Struktur, Gruppen, Benutzer),
- eine konsistente Ausgangsbasis für jeden Testlauf garantiert ist.

12.2.8 Automatisiertes Zurücksetzen der LDAP-Testumgebung

Da durch die Ausführung der Testfälle Benutzer im LDAP-System angelegt, aktualisiert oder gelöscht werden, musste nach jedem Testlauf ein definierter Ausgangszustand wiederhergestellt werden. Um diesen Prozess zu vereinfachen, wurde ein Bash-Skript verwendet, das die Testumgebung automatisiert neu aufsetzt.

Herkunft und Motivation

Das Skript wurde mit Hilfe von **ChatGPT (OpenAI)** generiert und anschließend in das Projekt übernommen. Es basiert funktional auf den Anweisungen im `README.md` der Entwicklungsumgebung, in dem ursprünglich alle Schritte manuell beschrieben waren - wie z. B. das Stoppen und Starten der Container sowie der Import von Schema- und Beispieldaten. Ziel war es, diese manuellen Kommandos in ein einziges, robustes Skript zu überführen.

Zielsetzung

Das Skript stellt sicher, dass:

- der OpenLDAP-Container vollständig neu gestartet wird,
- die im Projekt enthaltenen LDIF-Dateien korrekt geladen werden (`schema.ldif`, `01_ou_structure.ldif`, `02_groups.ldif`, `03_users.ldif`),
- eine konsistente und saubere Ausgangsbasis für jeden Testlauf gegeben ist.

Skriptübersicht

Listing 12.32: Von ChatGPT generiertes Skript - Automatisierung der README-Kommandos

```
#!/bin/bash

set -e
set -o pipefail

# Git Bash Workaround
export MSYS_NO_PATHCONV=1

echo ">>> Stopping and removing containers and volumes..."
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
docker compose down -v

echo ">>> Starting containers..."
docker compose up -d

echo ">>> Waiting for OpenLDAP container to be ready..."
sleep 5

# Check container
if ! docker ps --format '{{.Names}}' | grep -q '^openldap$'; then
    echo " Error: OpenLDAP container 'openldap' not running."
    exit 1
fi

# Validate LDIF files
docker exec openldap ls -la /container/service/slapd/assets

# Import LDIF files
winpty docker exec openldap sh -c "ldapadd -Y EXTERNAL -H ldapi:/// -f /container/service/slapd/assets/schema.
ldif"
winpty docker exec openldap sh -c "ldapadd -Y EXTERNAL -H ldapi:/// -f /container/service/slapd/assets/01
_ou_structure.ldif"
winpty docker exec openldap sh -c "ldapadd -Y EXTERNAL -H ldapi:/// -f /container/service/slapd/assets/02_groups.
ldif"
winpty docker exec openldap sh -c "ldapadd -Y EXTERNAL -H ldapi:/// -f /container/service/slapd/assets/03_users.
ldif"

echo " LDAP setup completed successfully."
```

Nutzen

Durch die Verwendung dieses Skripts konnte eine stabile, wiederholbare und leicht startbare LDAP-Testumgebung geschaffen werden. Es diente während der gesamten Testphase zur Vorbereitung der Tests und ermöglichte die zuverlässige Rücksetzung des Systemzustands nach jeder Testausführung.

Beispielinhalte der LDIF-Dateien

Die LDIF-Dateien enthalten die für die Tests notwendige LDAP-Struktur, Gruppen und Testbenutzer. Im Folgenden sind exemplarische Auszüge aus den jeweiligen Dateien dargestellt.

Strukturdefinition - 01_ou_structure.ldif Diese Datei definiert die grundlegenden Organisationseinheiten innerhalb des LDAP-Verzeichnisses, z. B. für Benutzer oder Gruppen:

Listing 12.33: Auszug aus 01_ou_structure.ldif

```
dn: ou=people,dc=paxportal,dc=ch
objectClass: organizationalUnit
ou: people
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
dn: ou=groups,dc=paxportal,dc=ch
objectClass: organizationalUnit
ou: groups
```

Gruppendefinition - 02_groups.ldif Diese Datei legt LDAP-Gruppen an, denen Benutzer später zugewiesen werden:

Listing 12.34: Auszug aus 02_groups.ldif

```
dn: cn=BROKERPORTAL,ou=groups,dc=paxportal,dc=ch
objectClass: groupOfNames
cn: BROKERPORTAL
member: cn=placeholder,dc=paxportal,dc=ch
```

Testbenutzer - 03_users.ldif Diese Datei enthält vorab definierte Testnutzer, die für Integrationstests verwendet werden:

Listing 12.35: Auszug aus 03_users.ldif

```
dn: cn=Marco Frei,ou=people,dc=paxportal,dc=ch
objectClass: inetOrgPerson
objectClass: paxPeople
cn: Marco Frei
sn: Frei
givenName: Marco
mail: marco.frei@pax.ch
uid: 100
```

Zusammenfassung

Durch die Kombination aus Container-Neustart und gezieltem Import der LDIF-Dateien wird die Testumgebung für jede Ausführung in einen sauberen Ausgangszustand zurückgesetzt. Die Strukturdateien sind modular aufgebaut und ermöglichen eine klare Trennung zwischen OUs, Gruppen und Benutzern.

12.2.9 Verfügbarkeit der Applikation prüfen – globalSetup.ts

Bevor die automatisierten Tests gestartet werden, muss sichergestellt sein, dass die zu testende Applikation tatsächlich läuft und erreichbar ist. Dafür wurde ein globalSetup-Script implementiert, das vor jedem Testlauf von Playwright automatisch ausgeführt wird.

Ziel

Das Skript validiert, ob die SOAP-Schnittstelle unter `http://localhost:8080/ws` erreichbar ist. Sollte dies nicht der Fall sein, wird der Testlauf mit einem entsprechenden Fehler beendet.

Implementierung

Listing 12.36: globalSetup.ts – Prüfung der Erreichbarkeit der API

```
import { request } from '@playwright/test';
import { createGetUserStatusXML } from '../services/createXmlService';

export default async function globalSetup() {
  const checkPayload = createGetUserStatusXML({
    idpUserId: '000',
    requestId: '1',
    requestTime: '2024-12-01T00:00:00Z',
  });

  const context = await request.newContext();
  try {
    const res = await context.post('http://localhost:8080/ws', {
      headers: { 'Content-Type': 'text/xml;charset=UTF-8' },
      data: checkPayload,
    });

    if (res.status() >= 500) {
      throw new Error('API antwortet, aber mit Serverfehler.');
```

Einbindung in die Testkonfiguration

Das Skript wurde in der Playwright-Konfigurationsdatei `playwright.config.ts` eingebunden und wird automatisch vor jedem Testlauf ausgeführt:

Listing 12.37: Ausschnitt aus playwright.config.ts

```
export default defineConfig({
```

```
globalSetup: './api-tests/globalSetup.ts',  
...  
});
```

Nutzen

Durch diese Prüfung kann frühzeitig festgestellt werden, ob die Applikation betriebsbereit ist. Das spart Zeit und vermeidet fehlschlagende Tests durch technische Ursachen wie vergessene Containerstarts oder Portprobleme. Bei Nichterreichbarkeit wird die Testausführung mit einem klaren Fehlerbild abgebrochen.

12.3 Implementierung der CI/CD Pipeline

12.3.1 Erweiterung der CI/CD-Pipeline zur Ausführung automatisierter API-Tests

Zur Sicherstellung der Funktionsfähigkeit des gesamten Systems wurden automatisierte API-Tests mithilfe von Playwright in die bestehende GitHub Actions-Pipeline integriert. Ziel ist es, die korrekte Verarbeitung von SOAP-Anfragen wie `addUser`, `updateUser`, `deleteUser` und `getUserStatus` automatisiert zu prüfen.

Initialisierungsskript Um in der Pipeline eine konsistente Testumgebung sicherzustellen, wurde ein separates Initialisierungsskript `init-ldap.sh` erstellt. Dieses Skript stellt sicher, dass der OpenLDAP-Container vollständig gestartet ist und initialisiert anschließend die Datenbank mit einer vordefinierten Struktur sowie Testdaten. Dazu gehören die benutzerdefinierte Schema-Datei, die organisatorische Struktur, Gruppen und Beispielbenutzer.

Listing 12.38: Von ChatGPT generiertes Initialisierungsskript für LDAP-Daten in der CI/CD-Pipeline

```
#!/bin/bash  
  
set -e  
set -o pipefail  
  
echo ">>> Waiting for OpenLDAP container to be ready..."  
  
timeout=30  
elapsed=0  
  
until docker exec openldap ldapsearch -Y EXTERNAL -H ldapi:/// -b "" -s base "(objectclass=*)" > /dev/null 2>&1;  
do  
    sleep 1  
    elapsed=$((elapsed+1))  
    if [ "$elapsed" -ge "$timeout" ]; then  
        echo " OpenLDAP did not become ready within ${timeout} seconds."  
        exit 1  
    fi  
done
```

```
echo "OpenLDAP is ready."

docker exec openldap ls -la /container/service/slapd/assets
docker exec openldap ldapadd -Y EXTERNAL -H ldapi:/// -f /container/service/slapd/assets/schema.ldif
docker exec openldap ldapadd -Y EXTERNAL -H ldapi:/// -f /container/service/slapd/assets/01_ou_structure.ldif
docker exec openldap ldapadd -Y EXTERNAL -H ldapi:/// -f /container/service/slapd/assets/02_groups.ldif
docker exec openldap ldapadd -Y EXTERNAL -H ldapi:/// -f /container/service/slapd/assets/03_users.ldif

echo " LDAP setup completed successfully."
```

Erklärung der Pipeline-Schritte Der neue Pipeline-Job `api-tests` wird nach dem erfolgreichen Build gestartet. Die einzelnen Schritte werden im Folgenden erläutert:

1. **Code Checkout:** Der Quellcode des Projekts wird aus dem Repository geladen.
2. **Debug-Ausgabe (optional):** Ausgabe des aktuellen Verzeichnisinhalts zur Fehlersuche, z. B. falls Pfade nicht stimmen.
3. **Docker Compose Installation:** Da GitHub Actions kein Docker Compose vorinstalliert hat, wird dieses Werkzeug manuell installiert. Es wird später für das Starten von Applikation und LDAP verwendet.
4. **Node.js Setup:** Die für Playwright benötigte Node.js-Version (hier: Version 20) wird bereitgestellt.
5. **Installation der JavaScript-Abhängigkeiten:** Mit `npm ci` werden alle im Projekt definierten JavaScript-Module installiert, die für die Ausführung der Tests benötigt werden.
6. **JDK Setup:** Da die Applikation in Java entwickelt ist, wird das Java Development Kit in Version 17 eingerichtet.
7. **Build der Applikation:** Das Projekt wird mit `mvnw clean install` gebaut. Die Tests werden hierbei übersprungen, da sie separat mit Playwright abgedeckt werden.
8. **Start der benötigten Container:** Die Applikation sowie der LDAP-Server werden mittels `docker-compose up -d` gestartet.
9. **Verfügbarkeit prüfen:** Es wird mit einem einfachen `curl`-Befehl regelmäßig geprüft, ob die Applikation über ihren Heartbeat-Endpunkt erreichbar ist.
10. **LDAP mit Testdaten befüllen:** Das oben beschriebene Initialisierungsskript wird ausgeführt, um das LDAP-System für die Tests vorzubereiten.
11. **Ausführen der Playwright-Tests:** Die eigentlichen API-Tests werden mit dem Befehl `npm run test:api` ausgeführt.
12. **HTML Report:** Die ausgeführten API-Tests werden als HTML Report gespeichert.
13. **Aufräumen:** Unabhängig vom Testergebnis werden alle gestarteten Container am Ende wieder heruntergefahren.

Listing 12.39: GitHub Actions Job zur Ausführung der API-Tests

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
api-tests:
  needs: build
  runs-on: ubuntu-latest
  steps:
    -name: Check out code
      uses: actions/checkout@v4

    -name: Show current directory structure (debug)
      run: |
        pwd
        ls -l

    -name: Install Docker Compose
      run: |
        sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(
          uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
        sudo chmod +x /usr/local/bin/docker-compose
        docker-compose version

    -name: Set up Node.js for Playwright
      uses: actions/setup-node@v4
      with:
        node-version: '20'

    -name: Install Node.js dependencies
      run: npm ci
      working-directory: ${github.workspace}

    -name: Set up JDK
      uses: actions/setup-java@v4
      with:
        distribution: 'temurin'
        java-version: '17'

    -name: Build project
      run: chmod +x ./mvnw && ./mvnw clean install -DskipTests
      working-directory: ${github.workspace}

    -name: Start Docker services
      run: docker-compose up -d
      working-directory: ${github.workspace}

    -name: Wait for app and LDAP to be ready
      run: |
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
-name: Wait for app and LDAP to be ready
run: |
  until curl -s -o /dev/null -w "${http_code}" http://localhost:8080/api/heartbeat | grep -q "
    200"; do
    echo "Waiting for app and LDAP to be ready..."
    sleep 2
  done
  echo "App and LDAP are ready!"

-name: Initialize LDAP with test data
run: bash ./scripts/init-ldap.sh
working-directory: ${github.workspace}

-name: Run Playwright API tests
run: npm run test:api
working-directory: ${github.workspace}

-name: Upload Playwright Report
if: always()
uses: actions/upload-artifact@v4
with:
  name: playwright-report
  path: playwright-report

-name: Shutdown and cleanup
if: always()
run: docker-compose down -v
working-directory: ${github.workspace}
```

Automatisierte Ausführung durch Cronjob

Damit die Tests auch unabhängig von einem push oder commit ausgeführt werden, wurde die Pipeline um einen zeitbasierten Auslöser (*Cronjob*) erweitert. Dadurch wird der komplette Workflow automatisch zweimal täglich gestartet – einmal morgens um 08:00 Uhr und einmal nachmittags um 16:00 Uhr (MEZ). Die Konfiguration befindet sich im oberen Bereich der Pipeline und sieht wie folgt aus:

Listing 12.40: Cronjob-Integration in der GitHub Actions Pipeline

```
on:
  push:
    branches:
      - main
  schedule:
    - cron: '0 6 * * *'
    - cron: '0 14 * * *'
```

workflow_dispatch:

Problem während der Entwicklung: Synchronisationsfehler zwischen Applikation und LDAP

Während der Entwicklung kam es wiederholt zu einem Problem in der CI/CD-Pipeline: Die automatisierten Tests wurden bereits gestartet, bevor die Applikation vollständig hochgefahren war oder bevor eine stabile Verbindung zwischen Applikation und dem OpenLDAP-Container bestand. Dies führte zu instabilen Testergebnissen oder Verbindungsfehlern.

Lösung: Heartbeat-Endpoint mit LDAP-Prüfung Zur Lösung dieses Problems wurde ein dedizierter HTTP-Endpoint `/api/heartbeat` in der Applikation implementiert. Dieser überprüft nicht nur, ob der Webserver läuft, sondern stellt explizit sicher, dass auch eine Verbindung zum LDAP-Server erfolgreich aufgebaut werden kann. Erst wenn diese Prüfung erfolgreich ist, startet die Pipeline mit den automatisierten Tests.

Die Implementierung verwendet den `LdapTemplate` von Spring, um eine einfache LDAP-Suchanfrage durchzuführen. Bei Erfolg antwortet der Endpoint mit HTTP 200 OK, andernfalls mit 503 Service Unavailable.

Listing 12.41: HeartbeatController zur Verfügbarkeitsprüfung von Applikation und LDAP

```
package ch.pax.ecohub.endpoints;

import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.ldap.core.AttributesMapper;
import org.springframework.ldap.core.LdapTemplate;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequiredArgsConstructor
public class HeartbeatController {

    private final LdapTemplate ldapTemplate;

    @GetMapping("/api/heartbeat")
    public ResponseEntity<String> getHeartbeat() {
        try {
            ldapTemplate.search(
                "",
                "(objectClass=*)",
                (AttributesMapper<Object>) attributes -> null
            );
            return ResponseEntity.ok("OK");
        } catch (Exception e) {
            return ResponseEntity.status(503).body("LDAP not reachable: " + e.getMessage());
        }
    }
}
```

Verwendung in der Pipeline In der `pipeline.yml` wurde ein Warte-Mechanismus eingebaut, der in regelmäßigen Abständen die Erreichbarkeit dieses Heartbeat-Endpunkts prüft. Erst wenn eine gültige HTTP 200-Antwort empfangen wird, wird mit der Initialisierung des LDAP und dem Ausführen der Tests fortgefahren. Dadurch wird sichergestellt, dass die Tests nur unter stabilen Bedingungen starten.

Listing 12.42: Warten auf Heartbeat-Endpunkt in der Pipeline

```
-name: Wait for app and LDAP to be ready
run: |
  until curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/api/heartbeat | grep -q "200";
  do
    echo "Waiting for app and LDAP to be ready..."
    sleep 2
  done
  echo "App and LDAP are ready!"
```

KAPITEL 13

TEST/QUALITÄTSSICHERUNG

Die Qualitätssicherung spielte im gesamten Projektverlauf eine zentrale Rolle. Ziel war es, sicherzustellen, dass alle implementierten Funktionen – insbesondere die LDAP-Zuordnung über die Mappingfunktion sowie die SOAP-Schnittstellen – korrekt, robust und nachvollziehbar arbeiten.

Dazu wurden sowohl manuelle als auch automatisierte Testverfahren eingesetzt. In der frühen Entwicklungsphase kam SOAP UI zum Einsatz, um gezielt einzelne Funktionen zu testen, insbesondere die korrekte Verarbeitung und Umsetzung von Nutzerdaten in der LDAP-Struktur.

Die nachfolgenden Abschnitte beschreiben die eingesetzten Testmethoden, deren Aufbau sowie ausgewählte Beispiele zur Veranschaulichung.

13.1 Test der Mapping-Funktion

Die korrekte Umsetzung der Mapping-Logik wurde zunächst mithilfe von SOAP UI manuell getestet. Dazu wurden gezielt Nutzerdaten mit unterschiedlichen Rollen (z. B. EL_ oder KL_) übermittelt, um zu überprüfen, ob die resultierenden LDAP-Einträge korrekt den entsprechenden Organisationseinheiten (OUs) und Gruppen zugeordnet wurden. Getestet wurde lokal gegen eine OpenLDAP-Instanz im Docker-Container.

13.1.1 Erster Test der Mapping-Funktion EL

Als erstes wurde sich ein Test-Nutzer ausgewählt, der keine Rolle im System hatte. Dieser hatte folgende Eigenschaften:

- `cn: Max Mustermann`
- `email: max.mustermann@pax.ch`
- `IDP-Nummer: 001`

- Rolle: keine

Nun wurde ein updateUser-Request an die SOAP-API geschickt, wobei die Rolle auf EL_ gesetzt wurde.

Listing 13.1: updateUser-Request für Test-Nutzer zur Mapping-Funktion EL

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:updateUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>001</idpUserId>
      <orgId>1</orgId>
      <igRoles>EL_OFFERT</igRoles>
      <orgRegNo>10</orgRegNo>
      ...
      <employed>true</employed>
      <givenName>Max</givenName>
      <surname>Mustermann</surname>
      <userEmail>max.mustermann@pax.ch</userEmail>
      ...
    </user:updateUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Als Antwort erhielt man folgende XML-Datei:

Listing 13.2: updateUser-Response für Test-Nutzer zur Mapping-Funktion

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonResponseType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
    </ns3:commonResponseType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Auf dem ersten Blick scheint der Request erfolgreich zu sein. Um dies zu überprüfen, wurde die LDAP-Struktur betrachtet.

Der Nutzer ist wie vorgesehen in der Gruppe Brokerdashboard zu finden.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



The image shows a screenshot of an LDAP configuration interface for a group named 'BROKERDASHBOARD'. The 'cn' field is set to 'BROKERDASHBOARD' and 'EXTERNPAX_A_BROKERDASHBOARD'. The 'member' field contains two entries: 'cn=admin,dc=paxportal,dc=ch' and 'uid=max.mustermann,ou=Makler,ou=people,dc=paxportal'. The interface includes buttons for '(add value)', '(rename)', '(add value)', and '(modify group members)'.

Abbildung 13.1: Test-Nutzer in der LDAP-Datenbank, in der Gruppe Brokerdashboard (Quelle: Eigenerarbeitung)

So auch in der Gruppe Brokerportal.

The image shows a screenshot of an LDAP configuration interface for a group named 'BROKERPORTAL'. The 'cn' field is set to 'BROKERPORTAL' and 'EXTERNPAX_A_BROKERPORTAL'. The 'member' field contains three entries: 'cn=admin,dc=paxportal,dc=ch', 'uid=hans.muster,ou=Mitarbeiter_GS,ou=people,dc=paxportal', and 'uid=max.mustermann,ou=Makler,ou=people,dc=paxportal'. The interface includes buttons for '(add value)', '(rename)', '(add value)', and '(modify group members)'.

Abbildung 13.2: Test-Nutzer in der LDAP-Datenbank, in der Gruppe Brokerportal (Quelle: Eigenerarbeitung)

Ebenso in der Gruppe Logismata.

The image shows a screenshot of an LDAP configuration interface for a group named 'LOGISMATA_B2B'. The 'cn' field is set to 'LOGISMATA_B2B' and 'EXTERNPAX_A_LOGISMATA_B2B'. The 'member' field contains three entries: 'cn=admin,dc=paxportal,dc=ch', 'uid=heidi.meier,ou=Mitarbeiter_AD,ou=people,dc=paxportal', and 'uid=max.mustermann,ou=Makler,ou=people,dc=paxportal'. The interface includes buttons for '(add value)', '(rename)', '(add value)', and '(modify group members)'.

Abbildung 13.3: Test-Nutzer in der LDAP-Datenbank, in der Gruppe Logismata (Quelle: Eigenerarbeitung)

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Und auch in der Gruppe Quicksale.

cn required, rdn

QUICKSALE *

EXTERNPAX_A_QUICKSALE *

(add value)

(rename)

member required

✗ cn=admin,dc=paxportal,dc=ch

➡ uid=heidi.meier,ou=Mitarbeiter_AD,ou=people,dc=paxpo

➡ uid=max.mustermann,ou=Makler,ou=people,dc=paxport

(add value)

(modify group members)

Abbildung 13.4: Test-Nutzer in der LDAP-Datenbank, in der Gruppe Quicksale (Quelle: Eigenerarbeitung)

Der Nutzer ist jedoch nicht in der Gruppe Vertragsservice zu finden, da diese Gruppe nicht zu den EL-Gruppen gehört.

cn required, rdn

VERTRAGSSERVICE *

EXTERNPAX_A_VERTRAGSSERVICE *

(add value)

(rename)

member requires

✗ cn=admin,dc=paxportal,dc=ch

➡ uid=hans.muster,ou=Mitarbeiter_GS,ou=people,dc=paxp

(add value)

(modify group members)

Abbildung 13.5: Test-Nutzer in der LDAP-Datenbank, nicht in der Gruppe Vertragsservice (Quelle: Eigenerarbeitung)

13.1.2 Zweiter Test der Mapping-Funktion KL

Um wirklich sicherzustellen dass die Mapping-Funktion korrekt arbeitet, wurde ein weiterer Test durchgeführt, dieses Mal mit der Rolle KL_. Es wurde der gleiche Test-Nutzer verwendet, um zu zeigen, dass die Mapping-Funktion auch bei einer Umstellung auf eine KL_-Rolle korrekt funktioniert, also den Nutzer den Kollektivleben-Gruppen zuordnet und ihn gleichzeitig aus den vorherigen EL_-Gruppen entfernt.

Listing 13.3: updateUser-Request für Test-Nutzer zur Mapping-Funktion KL

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
```


Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<user:updateUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <idpUserId>001</idpUserId>
  <orgId>1</orgId>
  <igRoles>KL_OFFERT</igRoles>
  <orgRegNo>10</orgRegNo>
  ...
  <employed>true</employed>
  <givenName>Max</givenName>
  <surname>Mustermann</surname>
  <userEmail>max.mustermann@pax.ch</userEmail>
  ...
</user:updateUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Als Antwort erhielt man folgende XML-Datei:

Listing 13.4: updateUser-Response für Test-Nutzer zur Mapping-Funktion

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonResponseType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
    </ns3:commonResponseType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Auch hier wurde in der LDAP-Datenbank die Gruppenzuordnung überprüft.

Der Nutzer ist nun in der Gruppe Vertragsservice zu finden.

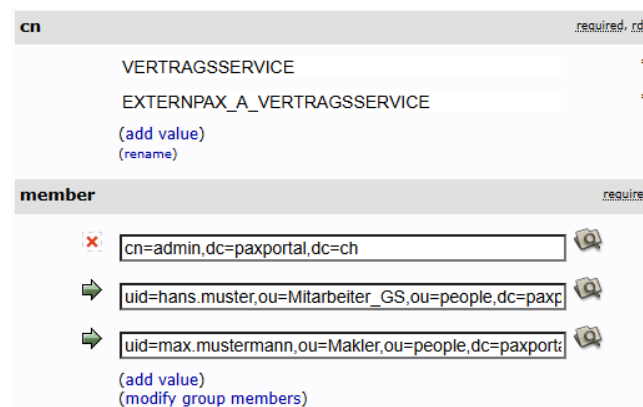


Abbildung 13.6: Test-Nutzer in der LDAP-Datenbank, in der Gruppe Vertragsservice (Quelle: Eigenerarbeitung)

Unter der Gruppe Brokerportal ist der Nutzer nicht mehr zu finden.

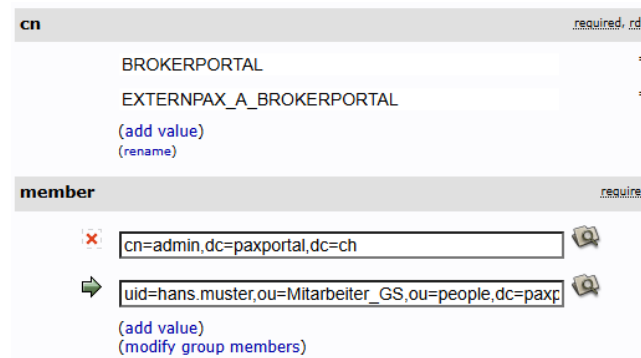


Abbildung 13.7: Test-Nutzer in der LDAP-Datenbank, nicht in der Gruppe Brokerportal (Quelle: Eigenerarbeitung)

13.1.3 Unit-Test der Mapping-Funktion

Auch die interne Logik zur Gruppenzuweisung und -entfernung auf LDAP-Ebene wurde mit Unit-Tests abgesichert. Ziel war es, sicherzustellen, dass die Methoden zur LDAP-Gruppenpflege korrekt und unabhängig vom Transportmedium (z. B. SOAP) funktionieren.

Ein Beispiel hierfür ist die Methode `removeUserFromAllGroups`, welche beim Update eines Nutzers alle bisherigen Gruppenzugehörigkeiten entfernt. Diese Methode wurde mit einem gezielten Unit-Test isoliert getestet. Dabei wurde das `LdapTemplate` mittels `Mockito` gemockt und überprüft, ob der Benutzer-DN in allen relevanten Gruppen korrekt entfernt wird.

Listing 13.5: Unit-Test für die Methode `removeUserFromAllGroups`

```
@Test
void removeUserFromAllGroups_shouldTryToRemoveUserFromEachGroup() {
    String userDn = "uid=testuser,ou=people";
    String fullUserDn = userDn + ",dc=paxportal,dc=ch";

    boolean result = personRepository.removeUserFromAllGroups(userDn);

    assertTrue(result);

    verify(ldapTemplate, atLeastOnce()).modifyAttributes(
        anyString(),
        argThat(mods ->
            Arrays.stream(mods)
                .anyMatch(mod ->
                    mod.getAttribute().getID().equals("member") &&
                    mod.getAttribute().contains(fullUserDn))
        )
    );
}
```

Auch die `updateUser`-Methode wurde mittels Unit-Test geprüft.

Listing 13.6: Unit-Test für die Methode updateUser

```
@Test
void updateUserGroup_shouldAddUserToCorrectGroups() {
    String userDn = "uid=testuser,ou=people";
    String fullUserDn = userDn + ",dc=paxportal,dc=ch";

    CompleteUserType payload = new CompleteUserType();
    payload.getIgRoles().add("EL_READ");
    payload.getIgRoles().add("KL_WRITE");

    personRepository.updateUserGroup(userDn, payload);

    // expected: add user to 2 groups (1 from EL_, 1 from KL_)
    verify(ldapTemplate, times(2)).modifyAttributes(
        anyString(),
        argThat(mods ->
            Arrays.stream(mods)
                .anyMatch(mod ->
                    mod.getAttribute().getID().equals("member") &&
                    mod.getAttribute().contains(fullUserDn))
                )
    );
}
```

Durch solche Unit-Tests wird die korrekte Verarbeitung der Logik unabhängig vom Aufrufer (z. B. dem SOAP-Webservice) überprüft. Dies erhöht sowohl die Wartbarkeit als auch die Robustheit der Anwendung.

Fazit: Die Tests der Mapping-Funktion zeigen, dass Nutzerdaten korrekt entsprechend ihrer Rolle den vorgesehenen LDAP-Gruppen zugeordnet werden. Auch eine Änderung der Rolle führt zur Anpassung der Gruppenzuordnung, wie beabsichtigt.

13.2 API-Tests mit Playwright

Zur Sicherstellung der Funktionalität der SOAP-Endpunkte (addUser, updateUser, deleteUser, getUserStatus) wurden automatisierte Tests mit Playwright implementiert. Dabei wird überprüft, ob die Rückgabe der API auf definierte Eingaben dem erwarteten Verhalten entspricht. Sowohl gültige als auch fehlerhafte Eingaben wurden berücksichtigt, um auch Randfälle abzudecken.

Für die lokale Ausführung der Tests wurde im package.json ein eigener Testbefehl hinterlegt:

Listing 13.7: Test-Befehl in package.json

```
"test:api": "npx playwright test --config=playwright.config.ts --workers=5"
```

Die Tests werden mit fünf parallelen Worker-Threads ausgeführt, was zu einer deutlich schnelleren Testdauer führt.

Verwendet wird die Konfigurationsdatei `playwright.config.ts`, in der die Testumgebung definiert ist. Sie legt unter anderem das globale Setup, das Testverzeichnis sowie die Basis-URL für die Testanfragen fest:

Listing 13.8: Konfiguration der Testumgebung (`playwright.config.ts`)

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  globalSetup: './api-tests/globalSetup.ts',
  testDir: './api-tests',
  timeout: 30000,
  retries: 2,
  use: {
    baseURL: 'http://localhost:8080',
    ignoreHTTPSErrors: true,
  },
});
```

Die Tests werden anschließend mit folgendem Befehl gestartet:

Listing 13.9: Ausführen der API-Tests

```
npm run test:api
```

Nach dem Start gibt Playwright das Testergebnis direkt in der Konsole aus. Ein erfolgreicher Testdurchlauf sieht wie folgt aus:

```
Running 16 tests using 4 workers

ok 1 api-tests\deleteUser.spec.ts:14:9 > SOAP API - deleteUser > TC10: deleteUser - IDP-Nummer erfolgreich gelöscht (101ms)
ok 2 api-tests\updateUser.spec.ts:36:9 > SOAP API - updateUser > TC6: updateUser - IDP Nummer erfolgreich aktualisiert (194ms)
ok 3 api-tests\addUser.spec.ts:37:9 > SOAP API - addUser > TC1: addUser - IDP Nummer erfolgreich vergeben (210ms)
ok 4 api-tests\deleteUser.spec.ts:30:9 > SOAP API - deleteUser > TC11: deleteUser - Payload unvollständig oder ungültig (31ms)
ok 5 api-tests\getUserStatus.spec.ts:12:9 > SOAP API - getUserStatus > TC14: getUserStatus - Nutzer erfolgreich gefunden (116ms)
ok 6 api-tests\deleteUser.spec.ts:48:9 > SOAP API - deleteUser > TC12: deleteUser - Nutzer nicht gefunden (38ms)
ok 7 api-tests\deleteUser.spec.ts:66:9 > SOAP API - deleteUser > TC13: deleteUser - no value (30ms)
ok 8 api-tests\updateUser.spec.ts:54:9 > SOAP API - updateUser > TC7: updateUser - Nutzer nicht gefunden (45ms)
ok 9 api-tests\getUserStatus.spec.ts:29:9 > SOAP API - getUserStatus > TC15: getUserStatus - Nutzer nicht gefunden (37ms)
ok 10 api-tests\addUser.spec.ts:58:9 > SOAP API - addUser > TC2: addUser - IDP Nummer bereits vergeben (36ms)
ok 11 api-tests\updateUser.spec.ts:73:9 > SOAP API - updateUser > TC8: updateUser - unvollständiger Payload (20ms)
ok 12 api-tests\updateUser.spec.ts:94:9 > SOAP API - updateUser > TC9: updateUser -no value (27ms)
ok 13 api-tests\getUserStatus.spec.ts:45:9 > SOAP API - getUserStatus > TC16: getUserStatus - no value (34ms)
ok 14 api-tests\addUser.spec.ts:79:9 > SOAP API - addUser > TC3: addUser - unvollständiger Payload (25ms)
ok 15 api-tests\addUser.spec.ts:103:9 > SOAP API - addUser > TC4: addUser - nicht existierende Email-Adresse (28ms)
ok 16 api-tests\addUser.spec.ts:125:9 > SOAP API - addUser > TC5: addUser - no value (28ms)

16 passed (1.7s)
```

Abbildung 13.8: Ergebnis der API-Tests in der Konsole (Quelle: Eigenerarbeitung)

Fazit: Die erfolgreiche Ausführung der Tests bestätigt, dass die implementierten SOAP-Endpunkte erwartungsgemäß reagieren und die definierten Rückgabewerte liefern.

13.3 CI/CD-Pipeline und Testintegration

Zur automatisierten Ausführung von Build- und Testprozessen wurde im Projekt eine GitHub-Actions-basierte CI/CD-Pipeline eingesetzt. Diese Pipeline wird bei jedem `push` ins Repository automatisch gestartet und führt die folgenden Schritte aus:

1. Kompilierung und Build der Applikation
2. Starten der Docker-Container für Applikation und OpenLDAP
3. Initialisierung des LDAP mit Testdaten
4. Ausführung der automatisierten API-Tests mit Playwright
5. Bereinigung der Umgebung

Die folgende Abbildung zeigt die vollständige Pipeline mit allen durchgeführten Schritten:

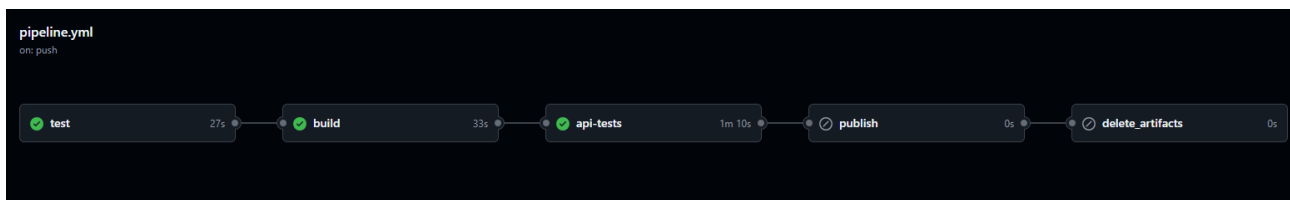


Abbildung 13.9: GitHub Actions – Übersicht der CI/CD-Pipeline (Quelle: Eigenerarbeitung)

Alle relevanten Aktionen wurden erfolgreich durchlaufen. Besonders wichtig ist dabei die Phase `api-tests`, in der die Playwright-Tests gegen die laufende Applikation ausgeführt werden. Auch hier liefen alle Tests fehlerfrei durch:

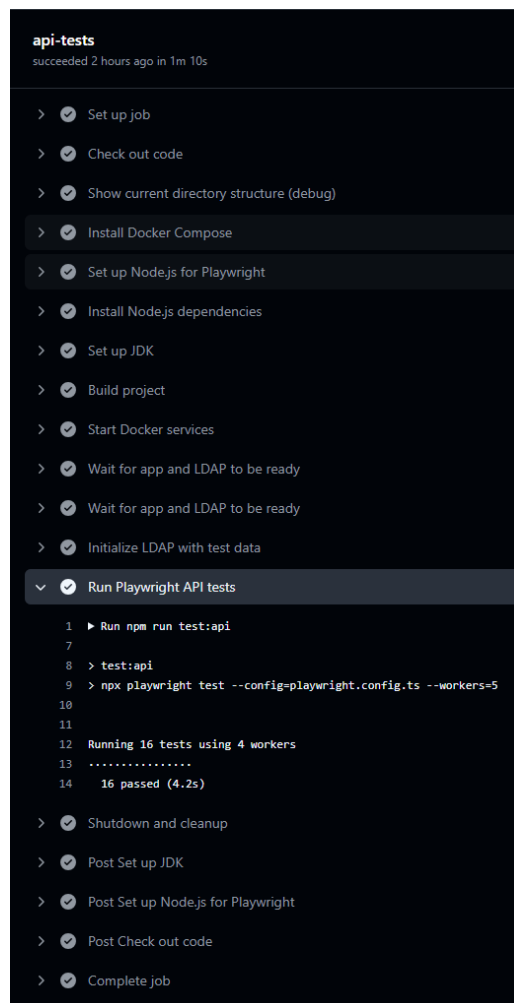


Abbildung 13.10: Erfolgreicher Playwright-Testlauf innerhalb der CI/CD-Pipeline (Quelle: Eigenerarbeitung)

13.3.1 Fehlermeldung bei nicht erreichbarem Service

Um sicherzustellen, dass die Playwright-Tests nur dann gestartet werden, wenn die Applikation vollständig hochgefahren ist, wurde ein `globalSetup`-Skript implementiert. Dieses überprüft vor Testbeginn, ob die SOAP-Schnittstelle unter `localhost:8080` erreichbar ist.

Ist dies nicht der Fall, wird der Testprozess abgebrochen und eine verständliche Fehlermeldung ausgegeben. Dadurch wird verhindert, dass Tests fälschlicherweise als fehlerhaft markiert werden, obwohl lediglich der Service noch nicht gestartet wurde.

```
> test:api
> npx playwright test --config=playwright.config.ts --workers=5
✖Service nicht erreichbar: localhost:8080
```

Abbildung 13.11: Fehlermeldung bei nicht erreichbarem Service (Quelle: Eigenerarbeitung)

13.3.2 Verifikation des Cronjobs

Um sicherzustellen, dass der geplante Cronjob korrekt funktioniert, wurde dieser während der Entwicklungsphase temporär so konfiguriert, dass der Workflow **jede Minute** ausgeführt wurde. Dies ermöglichte eine unmittelbare Überprüfung der automatischen Auslösung über die GitHub Actions-Oberfläche.

Nach erfolgreicher Validierung wurde das Cron-Intervall wieder auf die vorgesehenen Zeiten (täglich um 08:00 Uhr und 16:00 Uhr MEZ) zurückgestellt, um unnötige Ausführungen und Systembelastung zu vermeiden.

Die Testkonfiguration sah kurzzeitig wie folgt aus:

Listing 13.10: Temporäre Cronjob-Testkonfiguration

```
schedule:
  -cron: '*/1 * * * *' # alle 1 Minute
```

Diese Vorgehensweise stellte sicher, dass der automatisierte Ablauf verlässlich funktioniert, bevor er produktiv aktiviert wurde.

13.3.3 Hinweis zur README-Ergänzung

Zur Unterstützung anderer Entwicklerinnen und Entwickler wurde die Projekt-README.md-Datei um eine kurze Anleitung zur Ausführung der API-Tests ergänzt. Diese enthält Informationen zur lokalen Ausführung der Playwright-Testfälle sowie zur Anzeige des automatisch generierten Testreports.

Die Ergänzung sieht wie folgt aus:

Listing 13.11: Ausschnitt aus der erweiterten README.md

```
## API Tests with Playwright

The project includes a set of automated API tests written with [Playwright](https://playwright.dev/). These tests validate the functionality of the SOAP endpoints ('addUser', 'updateUser', 'deleteUser', 'getUserStatus') and ensure proper LDAP integration.

### Running the tests locally

To run the tests, ensure the application and the required services (e.g., OpenLDAP) are running via Docker:



```
''shell
docker-compose up -d
```


```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
'''

Then run the Playwright tests:
'''shell
npm install
npm run test:api
'''

### Test Resport
'''shell
npx playwright show-report
'''
```

Fazit: Die entwickelte CI/CD-Pipeline auf Basis von GitHub Actions ermöglichte eine vollständig automatisierte Prüfung der zentralen Projektkomponenten – vom Build über die Initialisierung der Testumgebung bis hin zur Ausführung und Verifikation der API-Tests. Durch den Einsatz von Docker und Playwright konnten realitätsnahe Tests in einer isolierten Umgebung durchgeführt werden.

Das zusätzlich implementierte `globalSetup`-Skript stellte sicher, dass die Tests nur bei tatsächlich erreichbarem Service ausgeführt wurden und somit verlässliche Ergebnisse lieferten. Auch die Testbarkeit geplanter Automatismen – wie der Cronjob – wurde nachvollziehbar sichergestellt.

Insgesamt trägt die Pipeline wesentlich zur Qualität, Wartbarkeit und Reproduzierbarkeit des Systems bei und bildet die Grundlage für zukünftige Erweiterungen und produktive Einsätze.

KAPITEL 14

ERGEBNISSE & AUSBLICK

In diesem Kapitel werden zunächst die zentralen Ergebnisse dieser Arbeit zusammengefasst. Dabei liegt der Fokus auf den funktionalen Erweiterungen und der technischen Umsetzung innerhalb der gegebenen Systemumgebung. Anschließend folgt ein Ausblick auf mögliche Weiterentwicklungen sowie Empfehlungen für den zukünftigen Einsatz und Ausbau der Lösung.

14.1 Ergebnisse der Umsetzung

Im Rahmen dieser Arbeit wurde die bestehende Funktionalität zur Benutzerpflege über die SOAP-Schnittstelle gezielt erweitert und technisch abgesichert.

Konkret wurde die `updateUser`-Funktion um eine rollenbasierte Mapping-Logik ergänzt. Diese ermöglicht es, Benutzer abhängig von den im XML definierten Rollen automatisch der passenden LDAP-Gruppe zuzuweisen. Die Logik berücksichtigt dabei vordefinierte Rollenmuster wie `EL_` oder `KL_` und weist entsprechende Gruppen in der LDAP-Struktur zu.

Zur Sicherstellung der Funktionalität wurde für alle vier Endpunkte (`addUser`, `updateUser`, `deleteUser`, `getUserStatus`) eine vollautomatisierte Testabdeckung mit Playwright entwickelt. Diese Tests lassen sich lokal ausführen und überprüfen sowohl die korrekte Verarbeitung gültiger Nutzerdaten als auch typische Fehlerfälle.

Darüber hinaus wurden die Tests in die bestehende GitHub-Actions-Pipeline integriert. Um sicherzustellen, dass die Tests erst bei verfügbarer LDAP-Verbindung ausgeführt werden, wurde ein Heartbeat-Mechanismus implementiert, der die Erreichbarkeit des Verzeichnisses vor dem Teststart überprüft.

Insgesamt wurde somit eine robuste, erweiterbare und automatisiert überprüfbare Grundlage für die weitere Integration der Benutzerverwaltung geschaffen.

14.2 Ausblick und Weiterentwicklung

Die umgesetzte Lösung bietet eine solide Grundlage für die automatisierte Verwaltung von Benutzern über eine SOAP-Schnittstelle mit LDAP-Anbindung. Dennoch bestehen mehrere sinnvolle Ansatzpunkte für eine zukünftige Weiterentwicklung.

Ein wesentliches Potenzial liegt in der Verfeinerung der bestehenden Mapping-Funktion. Derzeit basiert diese auf einfachen Präfixen wie `EL_` oder `KL_`. In einem nächsten Schritt könnte zusätzlich die Art der Berechtigung (z. B. Lese- oder Schreibrechte) berücksichtigt werden, um eine noch gezieltere Gruppenzuordnung zu ermöglichen.

Zudem sollte die Mapping-Logik nicht nur in der `updateUser`-Funktion, sondern auch in `addUser` integriert werden. So kann die Gruppenzuweisung bereits beim erstmaligen Anlegen eines Nutzers erfolgen und muss nicht nachträglich ergänzt werden. Darüber hinaus wäre es wünschenswert, dass `addUser` auch dann erfolgreich einen Eintrag erstellt, wenn beispielsweise die angegebene E-Mail-Adresse nicht im System bekannt ist – idealerweise mit einem Hinweis auf den fehlenden Abgleich.

Im Bereich der CI/CD-Pipeline bietet sich die Entwicklung einer eigenen GitHub Action an, mit der gezielte Testdaten als Input definiert und die entsprechenden XML-Antworten automatisiert validiert werden können. Das würde die Wartbarkeit und Nachvollziehbarkeit der Tests weiter verbessern.

Ein weiterer Entwicklungsschritt wäre die Ausführung der Tests nicht nur lokal, sondern auch innerhalb der Testumgebung. Aufgrund technischer Einschränkungen konnte dies im Rahmen dieser Arbeit noch nicht umgesetzt werden. In Zukunft sollte geprüft werden, ob dafür ein selbst gehosteter GitHub Runner eingesetzt werden kann oder ob eine alternative Lösung besser geeignet ist.

Diese Weiterentwicklungen würden die Lösung funktional abrunden und die Grundlage für einen produktiven Einsatz in komplexeren Szenarien schaffen.

Teil III

Anhang

HILFSMITTELVERZEICHNIS

- Hilfsmittel 1
- Hilfsmittel 2

ABKÜRZUNGSVERZEICHNIS

IPA Individuelle praktische Arbeit

GLOSSAR

- Begriff 1: Definition
- Begriff 2: Definition

ABBILDUNGSVERZEICHNIS

Abbildungsverzeichnis

2.1	Wasserfallmodell (Quelle: Eigenerarbeitung)	8
3.1	Projektaufbauorganisation (Quelle: Eigenerarbeitung)	11
5.1	Lokale Sicherung im Verzeichnisstruktur „Dokumente/IPA“	15
5.2	Tägliche Sicherungen auf Google Drive mit klarer Versionierung	15
5.3	Versionierung und Backup auf GitHub mit Commit-Historie	16
6.1	SOLL Zeitplan der Projektarbeit (Quelle: Eigenerarbeitung)	18
6.2	IST Zeitplan der Projektarbeit (Quelle: Eigenerarbeitung)	19
10.1	Rollenmodell auf der EcoHub-Plattform (Quelle: Eigenerarbeitung)	25
10.2	LDAP-Verzeichnisstruktur im lokalen Setup (Quelle: Eigenerarbeitung)	26
10.3	Business Process Model des IST-Zustands der Benutzerregistrierung (Quelle: Eigenerarbeitung)	27
10.4	Systemkontext (C1) - IST-Zustand der Benutzerverwaltung (Quelle: Eigenerarbeitung)	32
10.5	C2-Diagramm IST-Zustand - Containerübersicht (Quelle: Eigenerarbeitung)	33
10.6	C3-Komponentendiagramm - EcoHub Provisioning Service (Quelle: Eigenerarbeitung)	34
11.1	BPMN-Diagramm der updateUser-Funktion (Quelle: Eigenerarbeitung)	40
11.2	LDAP-Gruppen mit farblicher Zuordnung zu EL (gelb) und KL (rot) (Quelle: Eigenerarbeitung)	41
11.3	Pull Request mit strukturierter Commit-Historie im Branch ipa/mapping-funktion	60

12.1 Pfad zur application.yml-Datei (Quelle: Eigenerarbeitung)	63
13.1 Test-Nutzer in der LDAP-Datenbank, in der Gruppe Brokerdashboard (Quelle: Eigenerarbeitung)	97
13.2 Test-Nutzer in der LDAP-Datenbank, in der Gruppe Brokerportal (Quelle: Eigenerarbeitung)	97
13.3 Test-Nutzer in der LDAP-Datenbank, in der Gruppe Logismata (Quelle: Eigenerarbeitung)	97
13.4 Test-Nutzer in der LDAP-Datenbank, in der Gruppe Quicksale (Quelle: Eigenerarbeitung)	98
13.5 Test-Nutzer in der LDAP-Datenbank, nicht in der Gruppe Vertragsservice (Quelle: Eigenerarbeitung)	98
13.6 Test-Nutzer in der LDAP-Datenbank, in der Gruppe Vertragsservice (Quelle: Eigenerarbeitung)	99
13.7 Test-Nutzer in der LDAP-Datenbank, nicht in der Gruppe Brokerportal (Quelle: Eigenerarbeitung)	100
13.8 Ergebnis der API-Tests in der Konsole (Quelle: Eigenerarbeitung)	102
13.9 GitHub Actions – Übersicht der CI/CD-Pipeline (Quelle: Eigenerarbeitung)	103
13.10 Erfolgreicher Playwright-Testlauf innerhalb der CI/CD-Pipeline (Quelle: Eigenerarbeitung)	104
13.11 Fehlermeldung bei nicht erreichbarem Service (Quelle: Eigenerarbeitung)	105

TABELLENVERZEICHNIS

Tabellenverzeichnis

2.1 Vergleich zwischen Wasserfall und agilen Methoden	10
4.1 Aufgabenübersicht	12
10.1 Übersicht der zu testenden Endpunkte	27
11.1 Technische Arbeitspakete	57

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



QUELLENVERZEICHNIS